The topology of software risk in scientific models

3. Application to domain models

Arnald Puy

Contents

1 Preliminary

```
# PRELIMINARY FUNCTIONS ##########################################################
##################################################################################

sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",
                          "cowplot", "readxl", "ggrepel", "tidytext", "here",
                          "tidygraph", "igraph", "foreach", "parallel", "ggraph",
                          "tools", "purrr", "sensobol", "benchmarkme", "softwareRisk"))

# Select color palette ---------------------------------------------------------

color_languages <- c("fortran" = "steelblue", "python" = "lightgreen")

# Color to functional forms ----------------------------------------------------

color_functional_forms <- c("additive" = "#C65D09", "compensatory" = "#3B5B92")

# Set seed ---------------------------------------------------------------------

seed <- 123

# Source all .R files in the "functions" folder --------------------------------

r_functions <- list.files(path = here("functions"), pattern = "\\.R$", full.names = TRUE)
lapply(r_functions, source)
```

```
# CREATE DATASET #################################################################

# Path to folder ---------------------------------------------------------------

path <- "./datasets/call_metrics"

# List CSV files ---------------------------------------------------------------

files <- list.files(path, pattern = "\\.csv$", full.names = TRUE)

# Split by language ------------------------------------------------------------

python_files  <- grep("python",  files, value = TRUE, ignore.case = TRUE)
fortran_files <- grep("fortran", files, value = TRUE, ignore.case = TRUE)

base_fortran <- file_path_sans_ext(basename(fortran_files))
base_python <- file_path_sans_ext(basename(python_files))

model_names_fortran <- models <- sub(".*_", "", base_fortran)
model_names_python <- models <- sub(".*_", "", base_python)
```

```r
# Load and name files ------------------------------------------------------

python_list  <- lapply(python_files, fread)
fortran_list <- lapply(fortran_files, fread)

names(python_list) <- model_names_python
names(fortran_list) <- model_names_fortran

# RBIND --------------------------------------------------------------------

make_callgraph <- function(lst, lang) {
  rbindlist(lst, idcol = "model") %>%
    .[, language := lang] %>%
    .[, .(file, model, language, `function`, call)] %>%
    setnames(., c("function", "call"), c("from", "to"))
}

python_callgraphs  <- make_callgraph(python_list,  "python")
fortran_callgraphs <- make_callgraph(fortran_list, "fortran")

all_callgraphs <- rbind(python_callgraphs, fortran_callgraphs)

# SOURCE CODE CLASSIFICATION BY FUNCTIONAL ROLE ##################################

# Strip leading "./models/"------------------------------------------------
all_callgraphs[, file_clean:= sub("^\\./models/", "", file)]

# Provenance: file must be inside ./models to be eligible at all-----------
all_callgraphs[, in_model_tree:= !is.na(file) & nchar(file) > 0L &
                 grepl("^\\./models/", file)]

# Rest = everything after first segment------------------------------------
all_callgraphs[, rest:= sub("^[^/]+/", "", file_clean)]

# If rest starts with model_id/ then drop it (this is the duplicate)-------
all_callgraphs[, rest:= fifelse(startsWith(rest, paste0(tolower(model), "/")),
                                sub("^[^/]+/", "", rest), rest)]

# In case of triple nesting like vic/vic/vic/...---------------------------
all_callgraphs[, rest:= fifelse(startsWith(rest, paste0(tolower(model), "/")),
                                sub("^[^/]+/", "", rest), rest)]

# Top_level----------------------------------------------------------------
all_callgraphs[, top_level:= tstrsplit(rest, "/", fixed = TRUE, keep = 1L)]

# Useful generic external patterns (works across models)-------------------
all_callgraphs[, is_generic_external:= in_model_tree & (
```

```
  grepl("(^|/)(extern|external|externals|third[_-]?party|vendor|vendored)(/|$)",
        rest, ignore.case = TRUE) |
    grepl("(^|/)\\.lib(/|$)", rest, ignore.case = TRUE))]

# CTSM-specific external (framework + FoX parser under cdeps + CESM shared infra)
all_callgraphs[, is_ctsm_external:= (model == "CTSM") & in_model_tree & (
  grepl("^cime(/|$)", rest) |
    grepl("^cime_config(/|$)", rest) |
    grepl("^components/cdeps(/|$)", rest) |      # FoX XML/SAX parser
    grepl("(^|/)share_esmf(/|$)", rest) |        # shared ESMF infrastructure
    grepl("^src/unit_test_shr(/|$)", rest)       # unit tests
)]

# Extra CTSM allowlist (tightens "model core" to land-model code)---------------
CTSM_STRICT <- TRUE

all_callgraphs[, is_ctsm_allowed:= TRUE]

if (CTSM_STRICT) {
  all_callgraphs[model == "CTSM" & in_model_tree, is_ctsm_allowed :=
                   grepl("^src(/|$)", rest) |
                   grepl("^lilac(/|$)", rest) |
                   grepl("^components/cism(/|$)", rest)]
}

## Component classification (order matters)-------------------------------------
all_callgraphs[nchar(file) == 0L | is.na(file), component:= NA_character_]

all_callgraphs[!is.na(file) & nchar(file) > 0L, component:= fcase(

  # Anything not in ./models is external immediately
  !in_model_tree, "external_lib",

  # Generic external dirs (vendored/third_party/etc.)
  is_generic_external, "external_lib",

  # CI and vendored libraries
  top_level == ".github", "ci_cd",
  top_level == ".lib" | grepl("/\\.lib/", rest), "vendored_lib",

  # CIME / CESM / CDEPS infrastructure (framework)
  grepl("^cime/CIME/", rest), "framework",
  grepl("^cime_config/", rest), "framework",
  grepl("^cime/doc/", rest), "framework",
  grepl("^components/cdeps/cime_config/", rest), "framework",

  # CTSM-specific: treat cime + cdeps (incl FoX) as external/framework
```

```
  is_ctsm_external, "framework",

  # CTSM shared "shr_*" routines (framework by symbol)
  (model == "CTSM") & (grepl("^shr_", from) | grepl("^shr_", to)), "framework",

  # Tests (incl. SystemTests, case-insensitive)
  grepl("SystemTests/", rest, ignore.case = TRUE), "tests",
  grepl("/tests?/|^tests?/", rest, ignore.case = TRUE), "tests",
  grepl("(^|/)unit_test", rest, ignore.case = TRUE), "tests",

  # Drivers: Fortran code clearly in drivers directories
  grepl("(^|/)drivers(/|$)", rest) & language == "fortran", "driver",

  # Couplers: NUOPC / LILAC / CESM / cpl
  grepl("cpl_|/cesm/|/cpl/|/cpl_", rest), "coupler",

  # CLI / tools: scripts, setup, CTSM python utilities
  grepl("tools/|scripts/|cli\\.py$|setup\\.py$", rest), "cli_or_tool",
  grepl("^python/ctsm/", rest), "cli_or_tool",

  # Everything else: model core
  default = "model_core"
)]

# include flag for risk calculations ----------------------------------------
# Key changes:
# - require in_model_tree
# - exclude external_lib/framework/tests/etc (already via component filter)
# - CTSM strict allowlist (optional)
all_callgraphs[, include_in_risk := (
  in_model_tree &
    language %chin% c("fortran", "python") &
    component %chin% c("model_core", "coupler") &
    (model != "CTSM" | !CTSM_STRICT | is_ctsm_allowed)
)]

# --- Sanity check: should now be FALSE for the SAX/FoX functions in CTSM
all_callgraphs[
  model == "CTSM" & include_in_risk &
    (grepl("sax|parseDTD|parsefile|parsestring|runParser", from, ignore.case = TRUE) |
       grepl("sax|parseDTD|parsefile|parsestring|runParser", to,   ignore.case = TRUE)),
  .(from, to, file, rest, component)
]
```

```
## Empty data.table (0 rows and 5 cols): from,to,file,rest,component
```

```
# SUmmarize ------------------------------------------------------------------
```

```
all_callgraphs[, .N, component]
```

```
##       component     N
##          <char> <int>
## 1:        ci_cd     5
## 2: external_lib   100
## 3:    framework  8603
## 4:        tests   194
## 5:  cli_or_tool   820
## 6:   model_core 15727
## 7:       driver    64
## 8:      coupler   299
```

```
all_callgraphs[, .N, include_in_risk]
```

```
##    include_in_risk     N
##             <lgcl> <int>
## 1:           FALSE  9786
## 2:            TRUE 16026
```

```
# Remove module calls ------------------------------------------------

all_callgraphs <- all_callgraphs[!(from %in% "<module>")] %>%
  .[include_in_risk == TRUE]

# LOAD CYCLOMATIC COMPLEXITY VALUES FOR FUNCTIONS AND SUBROUTINES ##############

cc_unique <- fread("./datasets/cyclomatic_complexity_functions.csv")

# CREATE NETWORK FROM CALL GRAPHS ##############################################

all_graphs <- all_callgraphs[, .(graph = list(as_tbl_graph(.SD, directed = TRUE))),
                             model]

# ADD NODE METRICS #############################################################

# Define the weights to characterize risky nodes ---------------------------

alpha <- 0.6  # Weight to cyclomatic complexity
beta  <- 0.3  # Weight to in-degree (impact of bug upstream)
gamma <- 0.1  # Weight to betweenness (critical bridge)

# Add node metrics ---------------------------------------------------------

all_graphs[, graph:= Map(function(g, m) {

  comp_sub <- cc_unique[model == m]

  # mean cyclomatic complexity for this model & language ----------------
```

```
  mean_cyclo <- mean(comp_sub$cyclomatic_complexity, na.rm = TRUE)

  g %>%
    activate(nodes) %>%

    # Left join with dataset with cyclomatic complexity values -----------------

  left_join(comp_sub, by = "name") %>%

    # replace NA cyclomatic_complexity with model-language mean ----------------

  mutate(cyclomatic_complexity = if (!is.na(mean_cyclo)) {

    ifelse(is.na(cyclomatic_complexity), mean_cyclo, cyclomatic_complexity)

  } else {

    # if even the mean is NA (all NA in comp_sub), leave as-is

    cyclomatic_complexity
  }
  ) %>%

    # Remove Python MODULE_AGG / CLASS_AGG nodes from this graph
    # because they are not callable -------------------------------------------

  filter(!(language == "python" & type %in% c("MODULE_AGG", "CLASS_AGG"))) %>%

    # Calculation of key network metrics ---------------------------------------

  mutate(type = type,
         indeg = centrality_degree(mode = "in"),
         outdeg = centrality_degree(mode = "out"),
         btw = centrality_betweenness(directed = TRUE, weights = NULL),
         cyclo_sc = rescale(cyclomatic_complexity),
         indeg_sc = rescale(indeg),
         btw_sc = rescale(btw),
         risk_score = alpha * cyclo_sc * beta * indeg_sc * gamma * btw_sc)
},
graph, model)]

# EXTRACT NODE DF ##############################################################

all_graphs[, node_df:= lapply(graph, as_tibble, what = "nodes")]

# Export full node df ------------------------------------------------------
```

```r
full_node_df <- all_graphs %>%
  mutate(node_df = purrr::map(node_df, ~ select(.x, -model, -language))) %>%
  unnest(node_df) %>%
  select(-graph) %>%
  data.table()

write.xlsx(full_node_df, "full_node_df.xlsx")

# COMPUTE ALL PATHS AND THEIR RISK SCORES ######################################

# list of risk-form specifications ---------------------------------------------
# We run an additive and a compensatory version with p = -1

risk_specs <- list(additive     = list(p = 1),
                   compensatory = list(p = -1))

# Run programatically ----------------------------------------------------------

for (nm in names(risk_specs)) {
  spec <- risk_specs[[nm]]

  all_graphs[, (paste0("paths_tbl_", nm)):= Map(all_paths_fun, graph = graph,
                                                alpha = alpha,
                                                beta  = beta,
                                                gamma = gamma,
                                                p   = spec$p,
                                                complexity_col = "cyclomatic_complexity")]}

# Unnest tables ----------------------------------------------------------------

# For additive form -----------------

nodes_additive <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_additive", "nodes") %>%
  .[, risk_form := "additive"]
paths_additive <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_additive", "paths") %>%
  .[, risk_form := "additive"]

# For compensatory form ---------------

nodes_compensatory <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_compensatory", "nodes") %>%
  .[, risk_form := "compensatory"]
paths_compensatory <- unnest_paths_tbl_fun(all_graphs, "paths_tbl_compensatory", "paths") %>%
  .[, risk_form := "compensatory"]

# Rbind ------------------------------------------------------------------------

nodes_all <- rbind(nodes_additive, nodes_compensatory, fill = TRUE)
```

```
paths_all <- rbind(paths_additive, paths_compensatory, fill = TRUE)[, path_str:= NULL]

# Export ------------------------------------------------------------------

write.xlsx(nodes_all, "full_nodes_df.xlsx")
write.xlsx(paths_all, "full_paths_df.xlsx")
```

2 Uncertainty and sensitivity analysis

```
# CONDUCT UNCERTAINTY AND SENSITIVITY ANALYSIS ###################################

# Define sample size and order of effects ---------------------------------------

N <- 2^10
order <- "first"

# Run the function (we remove the vic and python model implementation because
# there are not paths) ----------------------------------------------------------

idx <- all_graphs[model != "VIC", which = TRUE]

# Define variants (input column -> output column + extra args) ------------------

ua_specs <- list(additive = list(paths_col = "paths_tbl_additive",
                                 out_col = "uncertainty_sensitivity_additive",
                                 extra = list()),
                 compensatory = list(paths_col = "paths_tbl_compensatory",
                                     out_col = "uncertainty_sensitivity_compensatory",
                                     extra = list()))

# Define parallel settings ------------------------------------------------------

ncores <- max(1L, parallel::detectCores() - 1L)

# Run UA/SA ---------------------------------------------------------------------

for (nm in names(ua_specs)) {
  spec <- ua_specs[[nm]]

  all_graphs[idx, (spec$out_col):= parallel::mclapply(X = get(spec$paths_col),
                                          FUN = function(x)
                                            do.call(uncertainty_fun,
                                                    c(list(all_paths_out = x,
                                                           N = N,
                                                           order = order),
                                                      spec$extra)),
```

```
                                        mc.cores = ncores
  )]
}

# Extract data ------------------------------------------------------------

# Additive ----------------------

nodes_us_additive <- unnest_us_fun(all_graphs,
                                   us_col = "uncertainty_sensitivity_additive",
                                   slot   = "nodes") %>%
  .[, risk_form:= "additive"]

paths_us_additive <- unnest_us_fun(all_graphs,
                                   us_col = "uncertainty_sensitivity_additive",
                                   slot = "paths") %>%
  .[, risk_form:= "additive"]

# Compensatory --------------------

nodes_us_compensatory <- unnest_us_fun(all_graphs,
                                       us_col = "uncertainty_sensitivity_compensatory",
                                       slot   = "nodes") %>%
  .[, risk_form:= "compensatory"]

paths_us_compensatory <- unnest_us_fun(all_graphs,
                                       us_col = "uncertainty_sensitivity_compensatory",
                                       slot   = "paths") %>%
  .[, risk_form:= "compensatory"]

# Rbind -------------------------------------------------------------------

nodes_us_all <- rbind(nodes_us_additive, nodes_us_compensatory, fill = TRUE)
paths_us_all <- rbind(paths_us_additive, paths_us_compensatory, fill = TRUE)

# Calculate median, q5 and q95 --------------------------------------------

cols <- c("uncertainty_analysis", "risk_trend", "gini_index")

for (nm in cols) {

  prefix <- switch(nm, uncertainty_analysis = "P", risk_trend = "risk",
                   gini_index = "gini")
  paths_us_all[, paste0(prefix, "_median"):= vapply(get(nm), median, numeric(1), na.rm = TRUE)]
  paths_us_all[, paste0(prefix, "_q5"):= vapply(get(nm), quantile, numeric(1), probs = 0.05, na
  paths_us_all[, paste0(prefix, "_q95"):= vapply(get(nm), quantile, numeric(1), probs = 0.95, n
```

```
}

# Extract sensitivity indices ------------------------------------------------

indices_all <- extract_sa_fun(nodes_us_all)
```

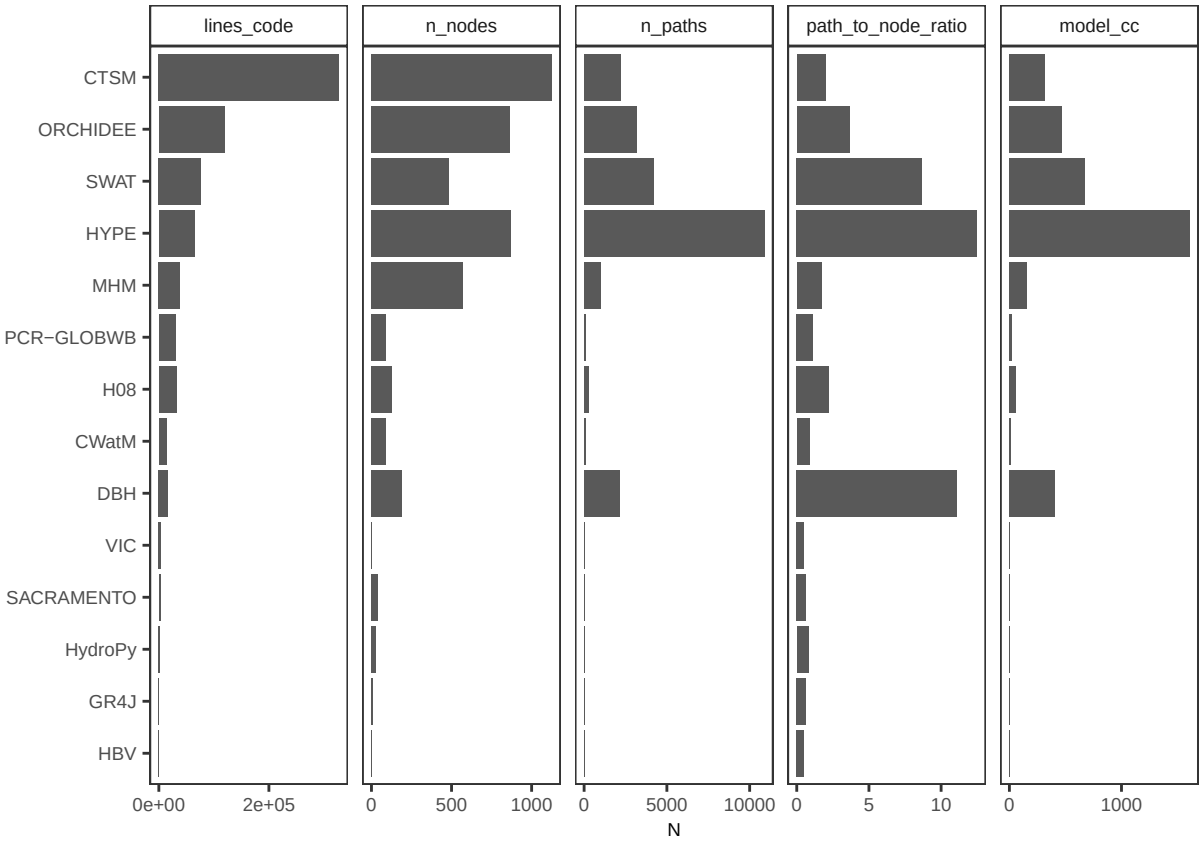
```
# CALCULATE SOME DESCRIPTIVE METRICS ###########################################

tmp <- data.table(paths_all[risk_form == "additive"])[, .(n_paths = .N), model] %>%
  .[order(-n_paths)]

tmp2 <- data.table(nodes_all[risk_form == "additive"])[, .(n_nodes = .N), model] %>%
  .[order(-n_nodes)]

# Path to node ratio: how interconnected the model is.
# Model_cc: Proxy for algorithmic complexity of model.
# Avg_path_length: Proxy for depth of dependency chains (risk-highway potential)
# Model fragility: more (error) propagation routes.
models_metrics <- merge(tmp, tmp2) %>%
  .[, `:=`(path_to_node_ratio = n_paths / n_nodes,
           model_cc = n_paths / log(n_nodes),
           avg_path_length = n_nodes / log(n_paths + 1),
           model_fragility_index = n_paths / (n_nodes * (n_nodes - 1)))]

models_metrics


# Read descriptive_stats_file ------------------------------------------------

num_cols <- c("files","functions","modules","lines","lines_code","lines_comments")

descriptive_stats <- data.table(read_xlsx("./datasets/descriptive_statistics/descriptive_statis

descriptive_stats <- dcast(melt(descriptive_stats, id.vars="model", measure.vars=num_cols),
                           model ~ variable, value.var ="value", fun.aggregate = function(z) su
  .[, lines_function := lines_code / functions]

all_descriptive_df <- merge(models_metrics, descriptive_stats)

# Sort by model ----------------------------------------------------------------
model_ordered <- all_descriptive_df[, sum(lines), model] %>%
  .[order(V1)]

# Plot descriptive measures per model ----------------------------------------

plot_descriptive <- melt(all_descriptive_df, measure.vars = c("lines_code", "n_nodes", "n_paths
                                                              "path_to_node_ratio", "model_cc"
```

```
  .[, model:= factor(model, levels = model_ordered[, model])] %>%
  ggplot(., aes(model, value)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(breaks = breaks_pretty(n = 2)) +
  scale_fill_manual(values = color_languages, name = "") +
  facet_wrap(~ variable, ncol = 7, scales = "free_x") +
  labs(x = "", y = "N") +
  theme_AP() +
  theme(legend.position = c(0.1, 0.3))
```

```
## Warning in melt.data.table(all_descriptive_df, measure.vars = c("lines_code", :
## 'measure.vars' [lines_code, n_nodes, n_paths, path_to_node_ratio, ...] are not
## all of the same type. By order of hierarchy, the molten data value column will
## be of type 'double'. All measure variables not of type 'double' will be coerced
## too. Check DETAILS in ?melt.data.table for more on coercion.
```

```
plot_descriptive
```

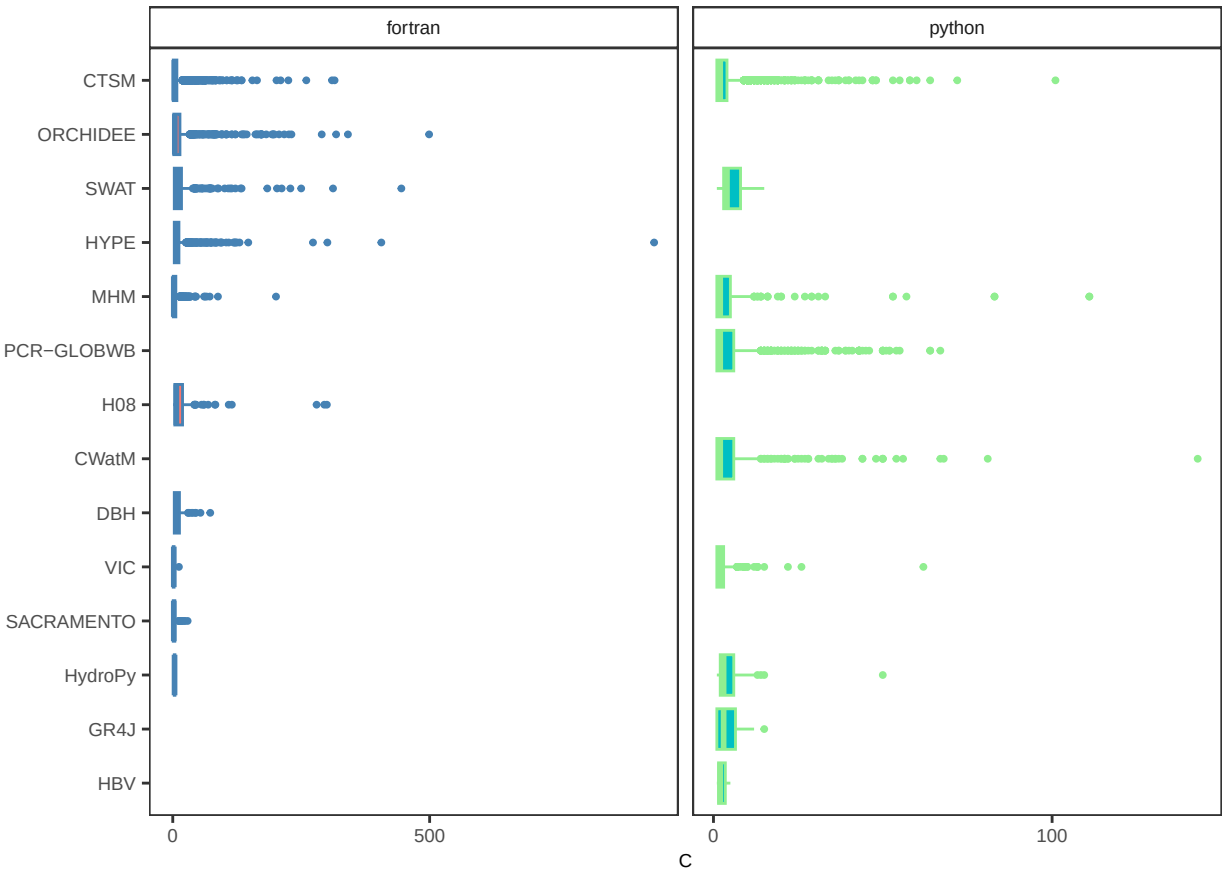
```
## Warning: No shared levels found between `names(values)` of the manual scale and the
## data's fill values.
```




```
# METRICS AT THE FILE AND FUNCTION LEVEL ########################################

folder <- "./datasets/results_per_function"
```

```r
# Get names of files ------------------------------------------------------

csv_files <- list.files(path = folder, pattern = "\\.csv$", full.names = TRUE)

# Split into file_metrics and func_metrics --------------------------------

file_metric_files <- grep("file_metrics", csv_files, value = TRUE)
func_metric_files <- grep("func_metrics", csv_files, value = TRUE)

# Build one named list ----------------------------------------------------

list_metrics <- list(file_metrics = setNames(lapply(file_metric_files, fread),
                                             basename(file_metric_files)),
                     func_metrics = setNames(lapply(func_metric_files, fread),
                                             basename(func_metric_files)))

# Create function to combine files ----------------------------------------

make_combined <- function(subset_list, pattern) {
  rbindlist(subset_list[grep(pattern, names(subset_list))], idcol = "source_file")
}

# Combine files -----------------------------------------------------------

metrics_combined <- list(file_fortran = make_combined(list_metrics$file_metrics, "fortran"),
                         file_python = make_combined(list_metrics$file_metrics, "python"),
                         func_fortran = make_combined(list_metrics$func_metrics, "fortran"),
                         func_python = make_combined(list_metrics$func_metrics, "python"))


# Functions to extract name of model and language from file ---------------

extract_model <- function(x)
  sub("^(file|func)_metrics_\\d+_([A-Za-z0-9-]+)_(fortran|python).*", "\\2", x)

extract_lang  <- function(x)
  sub("^(file|func)_metrics_\\d+_([A-Za-z0-9-]+)_(fortran|python).*", "\\3", x)

# Extract name of model and language --------------------------------------

metrics_combined <- lapply(metrics_combined, function(dt) {
  dt[, source_file:= sub("\\.csv$", "", basename(source_file))]
  dt[, model:= extract_model(source_file)]
  dt[, language:= extract_lang(source_file)]
  dt
})
```

```r
# Add column of complexity category ------------------------------------------

metrics_combined <- lapply(names(metrics_combined), function(nm) {
  dt <- as.data.table(metrics_combined[[nm]])
  if (grepl("^func_", nm) && "cyclomatic_complexity" %in% names(dt)) {
    dt[, complexity_category := cut(
      cyclomatic_complexity,
      breaks = c(-Inf, 10, 20, 50, Inf),
      labels = c("b1","b2","b3","b4")
    )]
  }
  dt
}) |> setNames(names(metrics_combined))

# Define labels ----------------------------------------------------------------

lab_expr <- c(b1 = expression(C %in% "(" * 0 * ", 10" * "]"),
              b2 = expression(C %in% "(" * 10 * ", 20" * "]"),
              b3 = expression(C %in% "(" * 20 * ", 50" * "]"),
              b4 = expression(C %in% "(" * 50 * ", " * infinity * ")"))

# Define vector to exclude classes that are not functions ----------------------

excluded_classes_vec <- c("MODULE_AGG", "CLASS_AGG")

# PLOT ########################################################################

plot_c_model <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x)
    x[, .(model, language, `function`, cyclomatic_complexity, loc, bugs, type)]) %>%
  rbindlist() %>%
  .[!type %in% excluded_classes_vec] %>%
  .[, model:= factor(model, levels = model_ordered[, model])] %>%
  na.omit() %>%
  ggplot(., aes(model, cyclomatic_complexity, fill = language, color = language)) +
  geom_boxplot(outlier.size = 0.7) +
  coord_flip() +
  scale_y_continuous(breaks = scales::breaks_pretty(n = 2)) +
  facet_wrap(~language, scales = "free_x") +
  labs(x = "", y = "C") +
  theme_AP() +
  scale_color_manual(values = color_languages) +
  theme(legend.position = "none",
        plot.margin = margin(0, 2, 0, 0))

plot_c_model
```

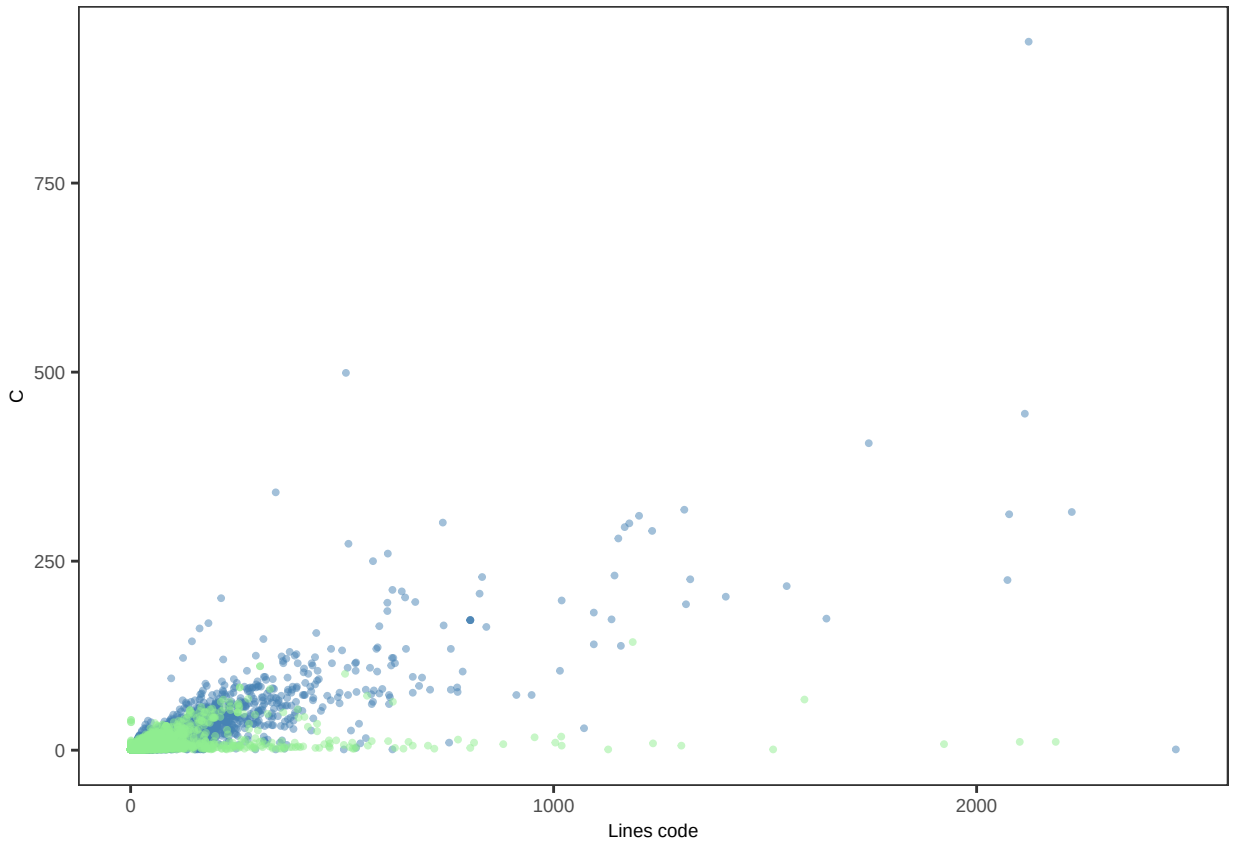



```
## ----plot_scatter_and_bar, dependson="read_metrics_function_data", fig.height=2.5, fig.width=

# Scatterplot cyclomatic vs lines of code ------------------------------------

plot_c_vs_loc <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x) x[, .(loc, cyclomatic_complexity, language)]) %>%
  rbindlist() %>%
  ggplot(., aes(loc, cyclomatic_complexity, color = language)) +
  geom_point(alpha = 0.5, size = 0.7) +
  scale_x_continuous(breaks = breaks_pretty(n = 3)) +
  labs(x = "Lines code", y = "C") +
  scale_color_manual(values = color_languages) +
  theme_AP() +
  scale_x_continuous(breaks = breaks_pretty(n = 2)) +
  theme(legend.position = "none")
```

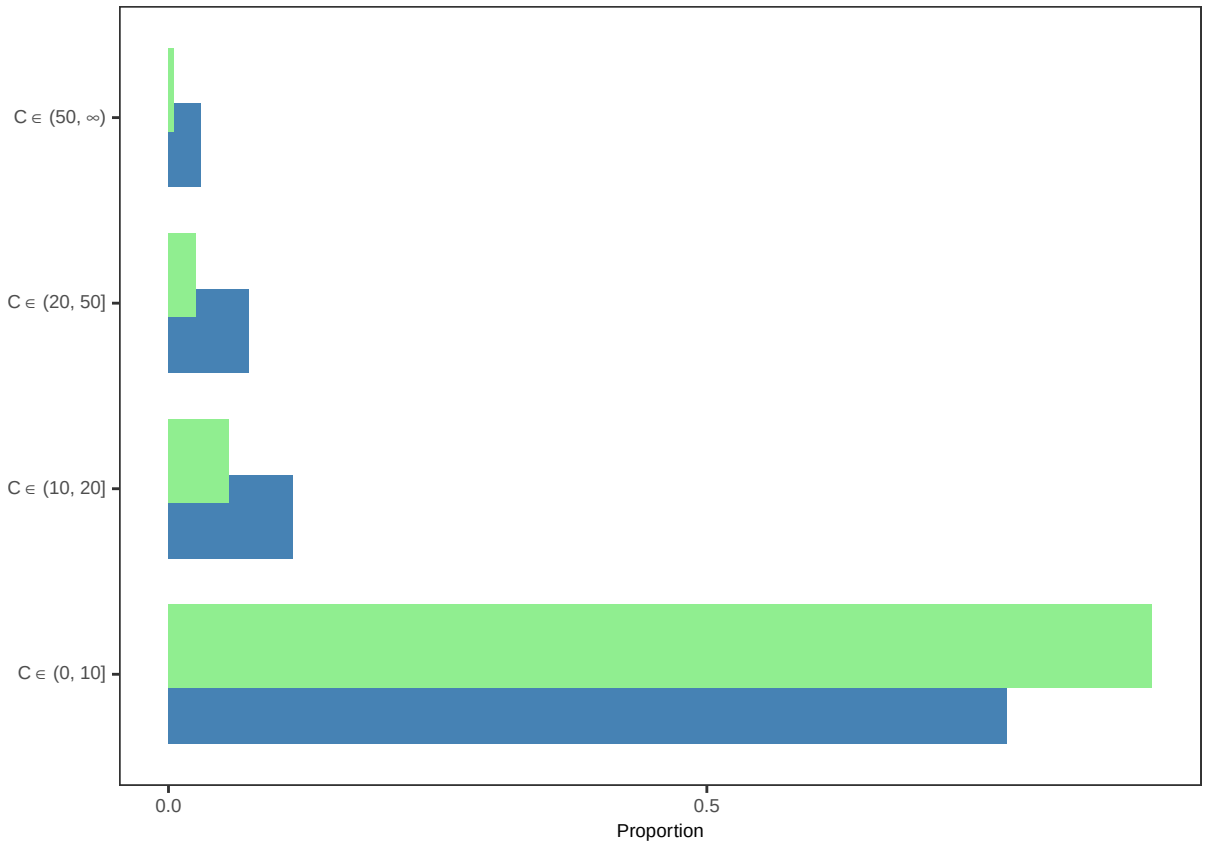
```
## Scale for x is already present.
## Adding another scale for x, which will replace the existing scale.
```

```
plot_c_vs_loc
```

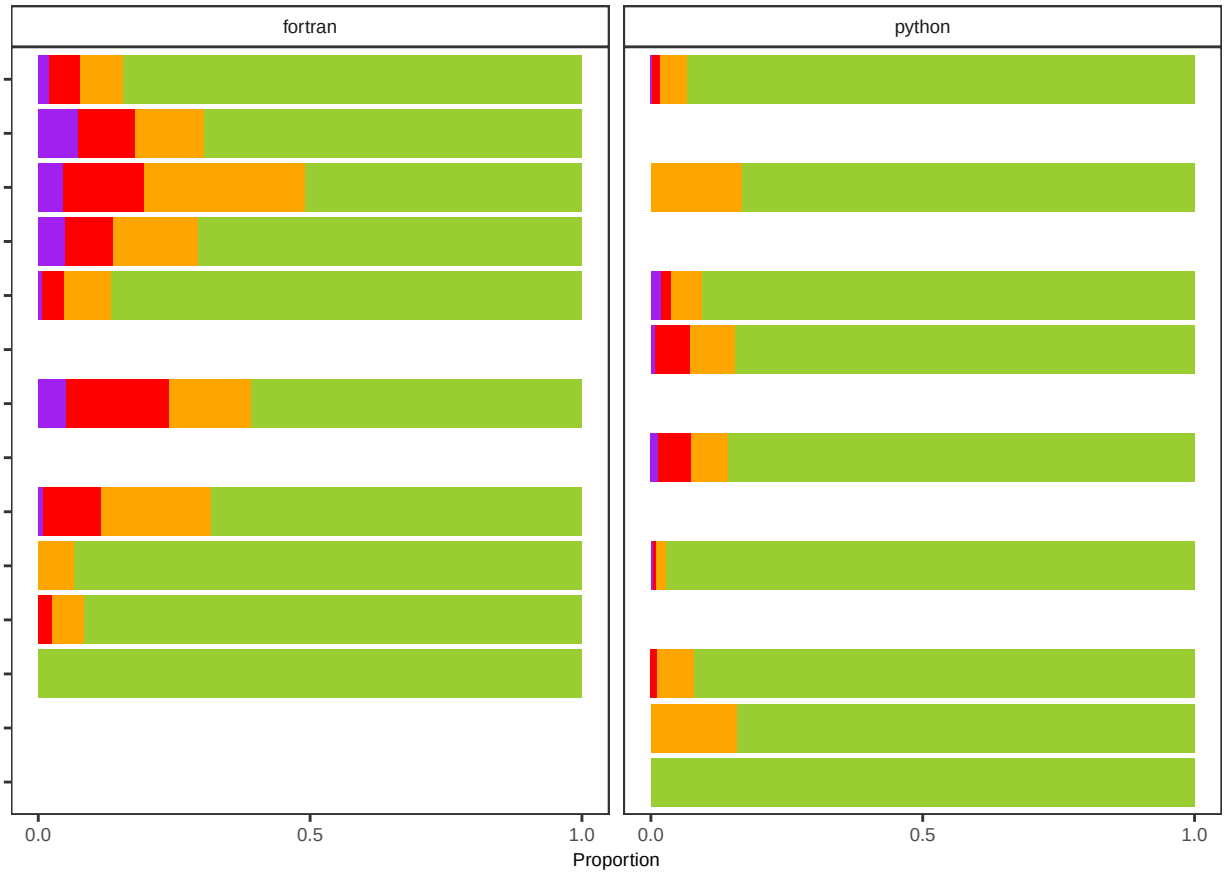
```
## Warning: Removed 1195 rows containing missing values or values outside the scale range
## (`geom_point()`).
```




```
# Count & proportion ------------------------------------------------------------

plot_bar_cyclomatic <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x) x[, .(complexity_category, language, type)]) %>%
  rbindlist() %>%
  .[!type %in% excluded_classes_vec] %>%
  .[, .N, .(complexity_category, language)] %>%
  .[, proportion := N / sum(N), language] %>%
  ggplot(., aes(complexity_category, proportion, fill = language)) +
  geom_bar(stat = "identity", position = position_dodge(0.6)) +
  scale_fill_manual(values = color_languages) +
  scale_y_continuous(breaks = scales::breaks_pretty(n = 3)) +
  scale_x_discrete(labels = lab_expr) +
  labs(x = "", y = "Proportion") +
  coord_flip() +
  theme_AP() +
  theme(legend.position = "none")

plot_bar_cyclomatic
```




```
plot_bar_category <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(., function(x)
    x[, .(model, language, complexity_category, type)]) %>%
  rbindlist() %>%
  .[!type %in% excluded_classes_vec] %>%
  .[, model:= factor(model, levels = model_ordered[, model])] %>%
  .[, .N, .(model, language, complexity_category)] %>%
  .[, proportion := N / sum(N), .(language, model)] %>%
  ggplot(., aes(model, proportion, fill = complexity_category)) +
  geom_bar(stat = "identity") +
  scale_fill_manual(values = c("yellowgreen", "orange", "red", "purple"),
                    labels = lab_expr,
                    name = "") +
  facet_wrap(~language) +
  labs(x = "", y = "Proportion") +
  coord_flip() +
  scale_y_continuous(breaks = scales::breaks_pretty(n = 3)) +
  theme_AP() +
  theme(legend.position = "none") +
  theme(axis.text.y = element_blank(),
        legend.text = element_text(size = 7),
        plot.margin = margin(0, 0, 0, 2))

plot_bar_category
```




```
# CORRELATION ANALYSIS AMONG METRICS ##############################################

# Compute pairwise Spearman correlations per model ------------------------------

cor_by_model <- full_node_df[, {

  dt <- .SD[, .(cyclomatic_complexity, indeg, btw)]
  dt <- na.omit(dt)

  if (nrow(dt) < 5) {

    NULL

  } else {

    pairs <- CJ(v1 = c("cyclomatic_complexity", "indeg", "btw"),
                v2 = c("cyclomatic_complexity", "indeg", "btw"))
    pairs <- pairs[v1 < v2]

    pairs[, .(
      v1 = v1, v2 = v2,
      rho = mapply(function(a, b)
```

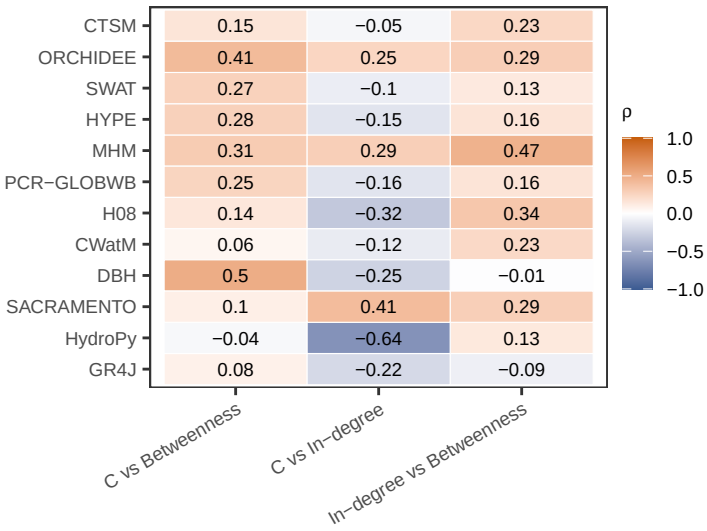
```
        cor(dt[[a]], dt[[b]], method = "spearman", use = "complete.obs"), v1, v2),
      n = nrow(dt)
    )]
  }
}, model]

# Rename metric pairs for readability ----------------------------------------

cor_by_model[, pair := fcase(
  v1 == "btw" & v2 == "cyclomatic_complexity", "C vs Betweenness",
  v1 == "btw" & v2 == "indeg", "In-degree vs Betweenness",
  v1 == "cyclomatic_complexity" & v2 == "indeg", "C vs In-degree"
)]

# Print summary table --------------------------------------------------------

dcast(cor_by_model, model ~ pair, value.var = "rho") %>%
  .[order(-`C vs In-degree`)] %>%
  print()
```

```
##          model C vs Betweenness C vs In-degree In-degree vs Betweenness
##         <char>            <num>          <num>                     <num>
##  1: SACRAMENTO       0.09677779     0.40511736                0.28808884
##  2:        MHM       0.30702429     0.28718493                0.46889455
##  3:   ORCHIDEE       0.41403730     0.24695587                0.29246428
##  4:       CTSM       0.15449318    -0.04979673                0.23387558
##  5:       SWAT       0.26992956    -0.10217372                0.12945208
##  6:      CWatM       0.05615097    -0.11506930                0.22656832
##  7:       HYPE       0.27695683    -0.15244085                0.16075698
##  8: PCR-GLOBWB       0.25490012    -0.15543808                0.16041661
##  9:       GR4J       0.08297398    -0.21546625               -0.08728716
## 10:        DBH       0.50147553    -0.25010020               -0.01119870
## 11:        H08       0.14286970    -0.31569313                0.33722469
## 12:    HydroPy      -0.04038146    -0.63815423                0.13430765
```

```
# Summary across all models -------------------------------------------------

cor_by_model[, .(mean_rho = round(mean(rho), 3),
                 median_rho = round(median(rho), 3),
                 min_rho = round(min(rho), 3),
                 max_rho = round(max(rho), 3)), pair]
```

```
##                       pair mean_rho median_rho min_rho max_rho
##                     <char>    <num>      <num>   <num>   <num>
## 1:         C vs Betweenness    0.210      0.205  -0.040   0.501
## 2: In-degree vs Betweenness    0.194      0.194  -0.087   0.469
## 3:           C vs In-degree   -0.088     -0.134  -0.638   0.405
```

```
# PLOT: heatmap of correlations per model and pair ----------------------------

cor_by_model[, model := factor(model, levels = model_ordered[, model])]

plot_metric_correlations <- ggplot(cor_by_model, aes(pair, model, fill = rho)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(rho, 2)), size = 2.5) +
  scale_fill_gradient2(low = "#3B5B92", mid = "white", high = "#C65D09",
                       midpoint = 0, limits = c(-1, 1),
                       name = expression(rho)) +
  labs(x = "", y = "") +
  theme_AP() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))

plot_metric_correlations
```




```
# POOLED CORRELATION (all functions across all models) ------------------------

pooled_cor <- full_node_df[, {
  dt <- na.omit(.SD[, .(cyclomatic_complexity, indeg, btw)])
  mat <- cor(dt, method = "spearman")
  data.table(
    CC_vs_indeg = mat["cyclomatic_complexity", "indeg"],
    CC_vs_btw = mat["cyclomatic_complexity", "btw"],
    indeg_vs_btw = mat["indeg", "btw"],
    n = nrow(dt)
  )
}]

cat("\nPooled Spearman correlations across all", pooled_cor$n, "functions:\n")
```

```
##
## Pooled Spearman correlations across all 4492 functions:
```

```
print(pooled_cor)
```

```
##    CC_vs_indeg CC_vs_btw indeg_vs_btw     n
##          <num>     <num>        <num> <int>
## 1:  0.06740364 0.2730882    0.2702837  4492
```

2.1 Analysis of metric robustness

```
# ROBUSTNESS: SWAP METRICS AND COMPARE PATH RANKINGS (OPTION 1) ###############
################################################################################

# Substitutions:
# A) C -> SLOC (lines of code)
# B) Betweenness -> Eigenvector centrality
# C) Both swapped simultaneously

# 1. Get SLOC per function from func_metrics -----------------------------------

loc_dt <- metrics_combined[grep("^func_", names(metrics_combined))] %>%
  lapply(function(x) x[, .(`function`, model, loc)]) %>%
  rbindlist() %>%
  setnames("function", "name") %>%
  na.omit() %>%
  .[, .(loc = mean(loc, na.rm = TRUE)), .(model, name)]

# Compute eigenvector centrality per model graph -------------------------------

eigen_dt <- all_graphs[, {
  g <- graph[[1]]

  # eigen_centrality can fail on some graph structures, so wrap in tryCatch
  ev <- tryCatch({
    igraph::eigen_centrality(g, directed = TRUE, scale = TRUE)$vector
  }, error = function(e) rep(NA_real_, igraph::vcount(g)))

  nd <- g %>%
    activate(nodes) %>%
    as_tibble(what = "nodes") %>%
    data.table()

  nd[, eigen := ev]
  nd[, .(name, eigen)]
}, model]

# Assemble node-level table with all metrics -----------------------------------

swap_nodes <- copy(full_node_df[, .(model, name, cyclomatic_complexity, indeg, btw)])
```

```r
swap_nodes <- merge(swap_nodes, loc_dt, by = c("model", "name"), all.x = TRUE)
swap_nodes <- merge(swap_nodes, eigen_dt, by = c("model", "name"), all.x = TRUE)

# Fill NAs with model-level means
swap_nodes[, `:=`(
  loc   = fifelse(is.na(loc),   mean(loc, na.rm = TRUE),   loc),
  eigen = fifelse(is.na(eigen), mean(eigen, na.rm = TRUE), eigen)
), model]

# Rescale within model and compute risk variants ----------------------------

swap_nodes[, `:=`(
  cc_sc    = rescale(cyclomatic_complexity),
  loc_sc   = rescale(loc),
  indeg_sc = rescale(indeg),
  btw_sc   = rescale(btw),
  eigen_sc = rescale(eigen)
), model]

swap_nodes[, `:=`(
  r_baseline  = alpha * cc_sc  + beta * indeg_sc + gamma * btw_sc,
  r_swap_cc   = alpha * loc_sc + beta * indeg_sc + gamma * btw_sc,
  r_swap_btw  = alpha * cc_sc  + beta * indeg_sc + gamma * eigen_sc,
  r_swap_both = alpha * loc_sc + beta * indeg_sc + gamma * eigen_sc
)]

# 5. Flatten paths to nodes and merge risk values ------------------------------

paths_base <- paths_all[risk_form == "additive"]
paths_long_swap <- paths_base[, .(node = unlist(path_nodes)), .(model, path_id)]

paths_long_swap <- merge(
  paths_long_swap,
  swap_nodes[, .(model, name, r_baseline, r_swap_cc, r_swap_btw, r_swap_both)],
  by.x = c("model", "node"), by.y = c("model", "name"),
  all.x = TRUE
)

# Replace NAs with 0 (unmatched nodes contribute no risk)
r_cols <- c("r_baseline", "r_swap_cc", "r_swap_btw", "r_swap_both")
for (rc in r_cols) paths_long_swap[is.na(get(rc)), (rc) := 0]

# Aggregate to path-level Pk = 1 - prod(1 - ri) ----------------------------

path_Pk_swap <- paths_long_swap[, .(
  Pk_baseline  = 1 - prod(1 - r_baseline),
  Pk_swap_cc   = 1 - prod(1 - r_swap_cc),
```

```
  Pk_swap_btw  = 1 - prod(1 - r_swap_btw),
  Pk_swap_both = 1 - prod(1 - r_swap_both)
), .(model, path_id)]

# Spearman rank correlations per model --------------------------------------

cor_swap <- path_Pk_swap[, .(
  rho_swap_cc   = cor(Pk_baseline, Pk_swap_cc,   method = "spearman"),
  rho_swap_btw  = cor(Pk_baseline, Pk_swap_btw,  method = "spearman"),
  rho_swap_both = cor(Pk_baseline, Pk_swap_both, method = "spearman"),
  n_paths = .N
), model]

print(cor_swap[order(-rho_swap_cc)])
```

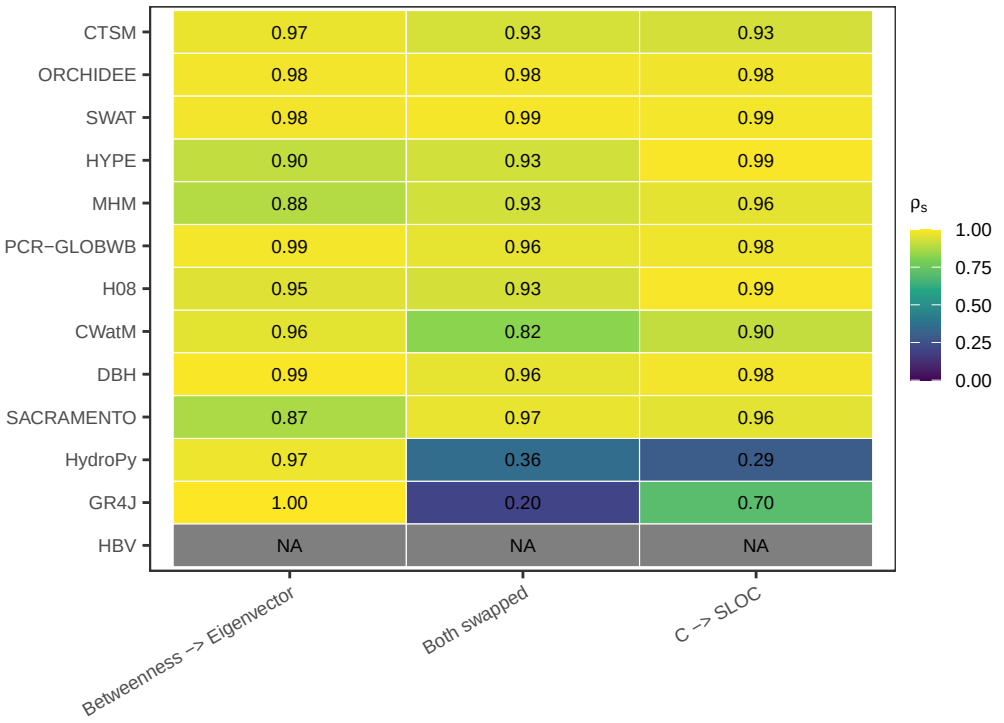
```
##          model rho_swap_cc rho_swap_btw rho_swap_both n_paths
##         <char>       <num>        <num>         <num>   <int>
##  1:       HYPE   0.9925671    0.9022826     0.9264122   10875
##  2:        H08   0.9892542    0.9500383     0.9321595     286
##  3:       SWAT   0.9852495    0.9838107     0.9879311    4169
##  4:        DBH   0.9831090    0.9931845     0.9624990    2123
##  5:   ORCHIDEE   0.9782099    0.9817711     0.9810912    3152
##  6: PCR-GLOBWB   0.9765959    0.9854109     0.9612748     101
##  7:        MHM   0.9595482    0.8809084     0.9260851     980
##  8: SACRAMENTO   0.9553878    0.8682224     0.9656829      26
##  9:       CTSM   0.9310340    0.9687674     0.9339693    2232
## 10:      CWatM   0.9042102    0.9568924     0.8200311      79
## 11:       GR4J   0.7000000    1.0000000     0.2000000       5
## 12:    HydroPy   0.2915725    0.9739074     0.3553540      21
## 13:        HBV          NA           NA            NA       1
```

```
# Top-10 overlap: how many of the top-10 paths are shared? ------------------

top_overlap_fun <- function(dt, k = 10) {
  dt[, {
    n <- min(k, .N)
    top_base <- path_id[order(-Pk_baseline)][1:n]
    top_cc   <- path_id[order(-Pk_swap_cc)][1:n]
    top_btw  <- path_id[order(-Pk_swap_btw)][1:n]
    top_both <- path_id[order(-Pk_swap_both)][1:n]
    .(overlap_cc   = length(intersect(top_base, top_cc))   / n,
      overlap_btw  = length(intersect(top_base, top_btw))  / n,
      overlap_both = length(intersect(top_base, top_both)) / n,
      n_compared = n)
  }, model]
}

overlap_10 <- top_overlap_fun(path_Pk_swap, k = 10)
```

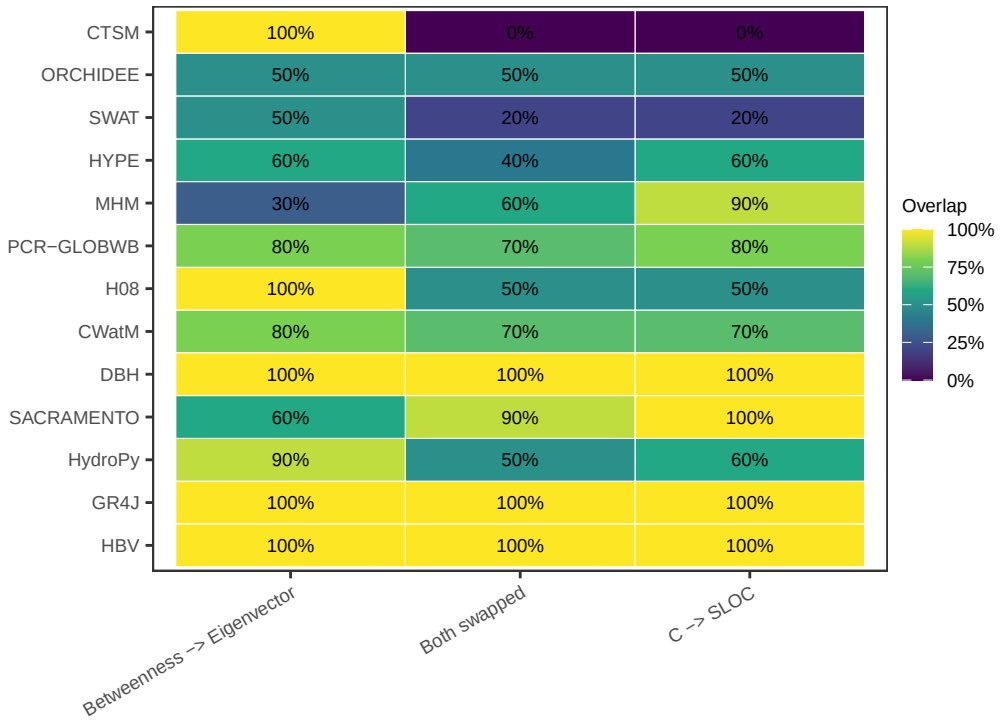
```
print(overlap_10[order(-overlap_cc)])
```

```
##          model overlap_cc overlap_btw overlap_both n_compared
##         <char>      <num>       <num>        <num>      <num>
##  1:        DBH        1.0         1.0          1.0         10
##  2:       GR4J        1.0         1.0          1.0          5
##  3:        HBV        1.0         1.0          1.0          1
##  4: SACRAMENTO        1.0         0.6          0.9         10
##  5:        MHM        0.9         0.3          0.6         10
##  6: PCR-GLOBWB        0.8         0.8          0.7         10
##  7:      CWatM        0.7         0.8          0.7         10
##  8:       HYPE        0.6         0.6          0.4         10
##  9:    HydroPy        0.6         0.9          0.5         10
## 10:        H08        0.5         1.0          0.5         10
## 11:   ORCHIDEE        0.5         0.5          0.5         10
## 12:       SWAT        0.2         0.5          0.2         10
## 13:       CTSM        0.0         1.0          0.0         10
```

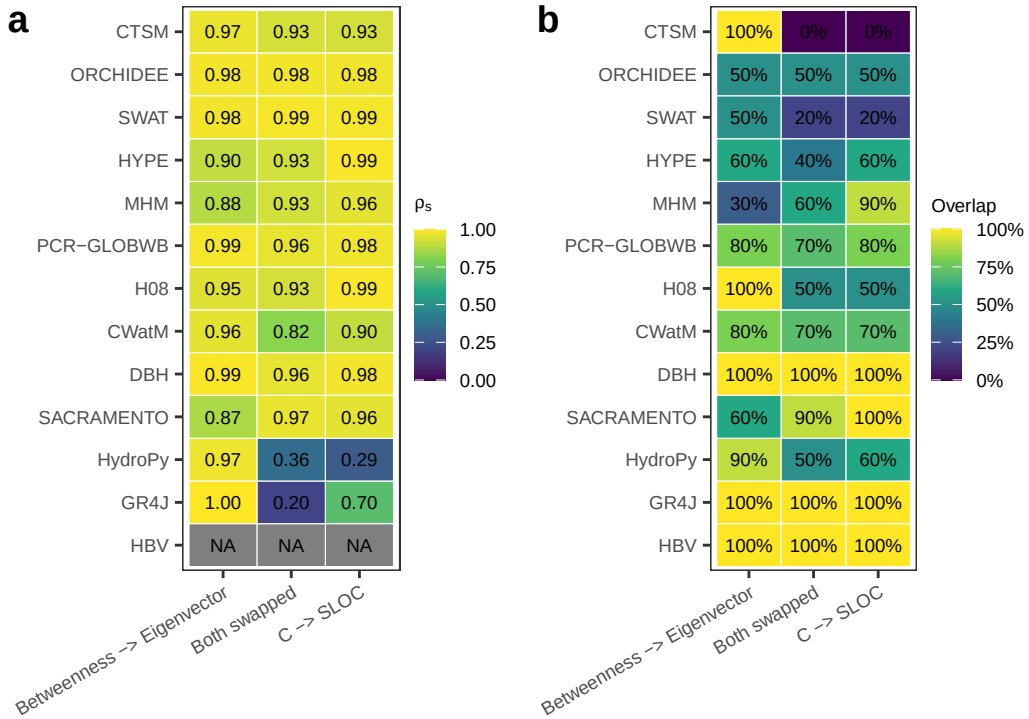
```
# Plot: correlation heatmap per model and substitution ----------------------

cor_long <- melt(cor_swap, id.vars = c("model", "n_paths"),
                 variable.name = "substitution", value.name = "rho")

cor_long[, substitution := fcase(
  substitution == "rho_swap_cc",   "C \u2192 SLOC",
  substitution == "rho_swap_btw",  "Betweenness \u2192 Eigenvector",
  substitution == "rho_swap_both", "Both swapped"
)]

cor_long[, model := factor(model, levels = model_ordered[, model])]

plot_swap_robustness <- ggplot(cor_long, aes(substitution, model, fill = rho)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", rho)), size = 2.5) +
  scale_fill_viridis_c(option = "D", direction = 1, limits = c(0, 1),
                       name = expression(rho[s])) +
  labs(x = "", y = "") +
  theme_AP() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))

plot_swap_robustness
```




```
# Plot: top-10 overlap heatmap ------------------------------------------

overlap_long <- melt(overlap_10, id.vars = c("model", "n_compared"),
                     variable.name = "substitution", value.name = "overlap")

overlap_long[, substitution := fcase(
  substitution == "overlap_cc",   "C \u2192 SLOC",
  substitution == "overlap_btw",  "Betweenness \u2192 Eigenvector",
  substitution == "overlap_both", "Both swapped"
)]

overlap_long[, model := factor(model, levels = model_ordered[, model])]

plot_overlap <- ggplot(overlap_long, aes(substitution, model, fill = overlap)) +
  geom_tile(color = "white") +
  geom_text(aes(label = scales::percent(overlap, accuracy = 1)), size = 2.5) +
  scale_fill_viridis_c(option = "D", direction = 1, limits = c(0, 1),
                       name = "Overlap", labels = scales::percent) +
  labs(x = "", y = "") +
  theme_AP() +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))

plot_overlap
```




```
# Merge both panels for supplementary figure ------------------------------

plot_grid(plot_swap_robustness, plot_overlap, ncol = 2,
          labels = c("a", "b"), align = "h", axis = "tb")
```




```
# OMNIBUS METRICS ACROSS ALL MODELS ##########################################
```

```r
# Rank correlations: summary across models ---------------------------------

omnibus_cor <- cor_long[, .(
  mean_rho   = round(mean(rho, na.rm = TRUE), 3),
  median_rho = round(median(rho, na.rm = TRUE), 3),
  min_rho    = round(min(rho, na.rm = TRUE), 3),
  max_rho    = round(max(rho, na.rm = TRUE), 3),
  sd_rho     = round(sd(rho, na.rm = TRUE), 3)
), substitution]

cat("\n--- Rank correlation summary ---\n")
```

```
##
## --- Rank correlation summary ---
```

```r
print(omnibus_cor)
```

```
##                 substitution mean_rho median_rho min_rho max_rho sd_rho
##                       <char>    <num>      <num>   <num>   <num>  <num>
## 1:                  C → SLOC    0.887      0.968   0.292   0.993  0.204
## 2: Betweenness → Eigenvector    0.954      0.971   0.868   1.000  0.045
## 3:              Both swapped    0.829      0.933   0.200   0.988  0.263
```

```r
# Top-10 overlap: summary across models ------------------------------------

omnibus_overlap <- overlap_long[, .(
  mean_overlap   = round(mean(overlap, na.rm = TRUE), 3),
  median_overlap = round(median(overlap, na.rm = TRUE), 3),
  min_overlap    = round(min(overlap, na.rm = TRUE), 3),
  max_overlap    = round(max(overlap, na.rm = TRUE), 3),
  sd_overlap     = round(sd(overlap, na.rm = TRUE), 3)
), substitution]

cat("\n--- Top-10 overlap summary ---\n")
```

```
##
## --- Top-10 overlap summary ---
```

```r
print(omnibus_overlap)
```

```
##                 substitution mean_overlap median_overlap min_overlap
##                       <char>        <num>          <num>       <num>
## 1:                  C → SLOC        0.677            0.7         0.0
## 2: Betweenness → Eigenvector        0.769            0.8         0.3
## 3:              Both swapped        0.615            0.6         0.0
##    max_overlap sd_overlap
##          <num>      <num>
## 1:           1      0.322
## 2:           1      0.243
## 3:           1      0.313
```

```r
# Weighted by number of paths (larger models count more) --------------------

omnibus_cor_weighted <- cor_long[, .(
  weighted_mean_rho = round(weighted.mean(rho, n_paths, na.rm = TRUE), 3)
), substitution]

omnibus_overlap_weighted <- overlap_long[, .(
  weighted_mean_overlap = round(weighted.mean(overlap, n_compared, na.rm = TRUE), 3)
), substitution]

cat("\n--- Path-weighted rank correlation ---\n")
```

```
##
## --- Path-weighted rank correlation ---
```

```r
print(omnibus_cor_weighted)
```

```
##                 substitution weighted_mean_rho
##                       <char>             <num>
## 1:                   C → SLOC             0.980
## 2: Betweenness → Eigenvector             0.941
## 3:              Both swapped             0.947
```

```r
cat("\n--- Path-weighted top-10 overlap ---\n")
```

```
##
## --- Path-weighted top-10 overlap ---
```

```r
print(omnibus_overlap_weighted)
```

```
##                 substitution weighted_mean_overlap
##                       <char>                 <num>
## 1:                   C → SLOC                 0.638
## 2: Betweenness → Eigenvector                 0.741
## 3:              Both swapped                 0.569
```

```r
# Where do "lost" top-10 paths end up under each substitution? ----------------

top10_fate <- path_Pk_swap[, {
  n <- min(10, .N)
  base_top <- path_id[order(-Pk_baseline)][1:n]

  rank_cc   <- frank(-Pk_swap_cc)
  rank_btw  <- frank(-Pk_swap_btw)
  rank_both <- frank(-Pk_swap_both)

  lost_cc   <- setdiff(base_top, path_id[order(-Pk_swap_cc)][1:n])
  lost_btw  <- setdiff(base_top, path_id[order(-Pk_swap_btw)][1:n])
  lost_both <- setdiff(base_top, path_id[order(-Pk_swap_both)][1:n])
```

```
  rbind(
    if (length(lost_cc) > 0)
      data.table(substitution = "C_to_SLOC",
                 path_id = lost_cc,
                 new_rank = rank_cc[match(lost_cc, path_id)]),
    if (length(lost_btw) > 0)
      data.table(substitution = "Btw_to_Eigen",
                 path_id = lost_btw,
                 new_rank = rank_btw[match(lost_btw, path_id)]),
    if (length(lost_both) > 0)
      data.table(substitution = "Both",
                 path_id = lost_both,
                 new_rank = rank_both[match(lost_both, path_id)])
  )
}, model]

# Summary: how far do lost paths fall? ----------------------------------------

top10_fate[, .(
  median_new_rank = median(new_rank),
  max_new_rank    = max(new_rank),
  n_lost          = .N
), substitution]
```

```
##    substitution median_new_rank max_new_rank n_lost
##          <char>           <num>        <num>  <int>
## 1:    C_to_SLOC            21.0       2065.5     42
## 2:         Both            22.0       2034.5     50
## 3: Btw_to_Eigen            14.5         27.0     30
```

```
# MERGE FIGURES ###############################################################

legend2 <- get_legend_fun(plot_bar_category + theme(legend.position = "top"))
```

```
## Warning: `is.ggplot()` was deprecated in ggplot2 3.5.2.
## i Please use `is_ggplot()` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

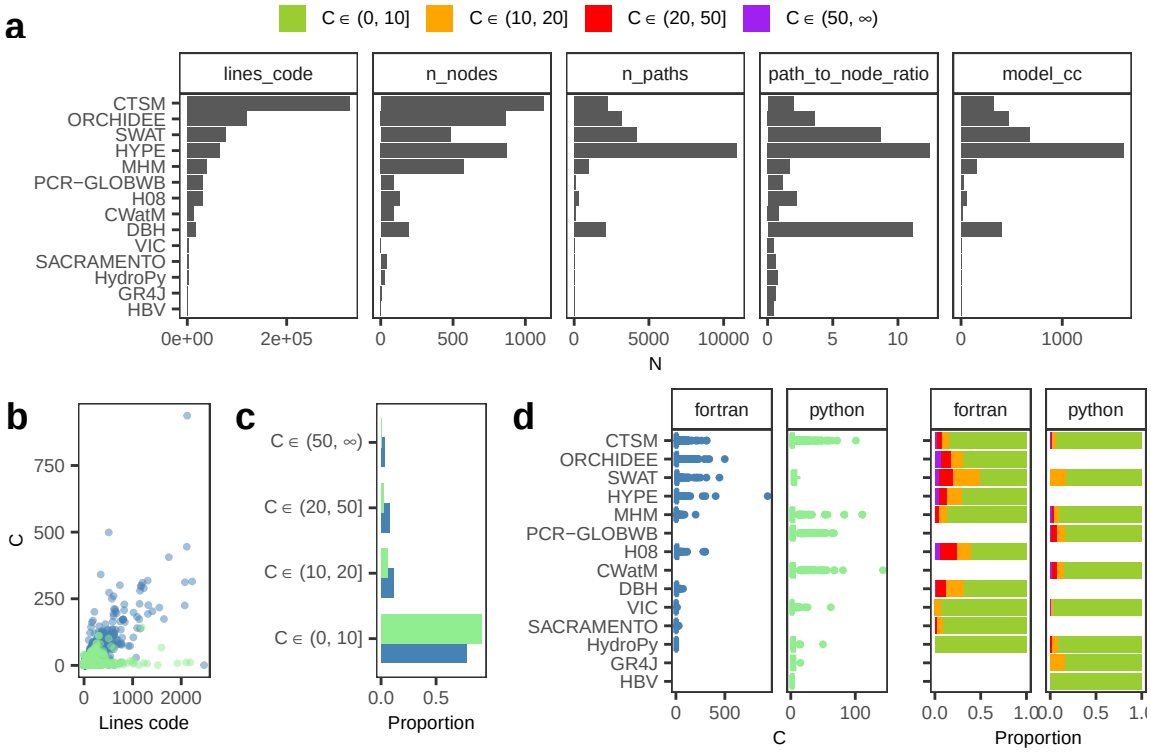
```
top_plot <- plot_grid(legend2, plot_descriptive, rel_heights = c(0.1, 0.9), ncol = 1,
                      labels = "a")
```

```
## Warning: No shared levels found between `names(values)` of the manual scale and the
## data's fill values.
```

```
bottom <- plot_grid(plot_c_vs_loc, plot_bar_cyclomatic, plot_c_model,
                    plot_bar_category, ncol = 4, rel_widths = c(0.2, 0.24, 0.34, 0.22),
                    labels = c("b", "c", "d"))
```

```
## Warning: Removed 1195 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

```
plot_grid(top_plot, bottom, ncol = 1, rel_heights = c(0.52, 0.48), align = "h",
          axis = "tb")
```



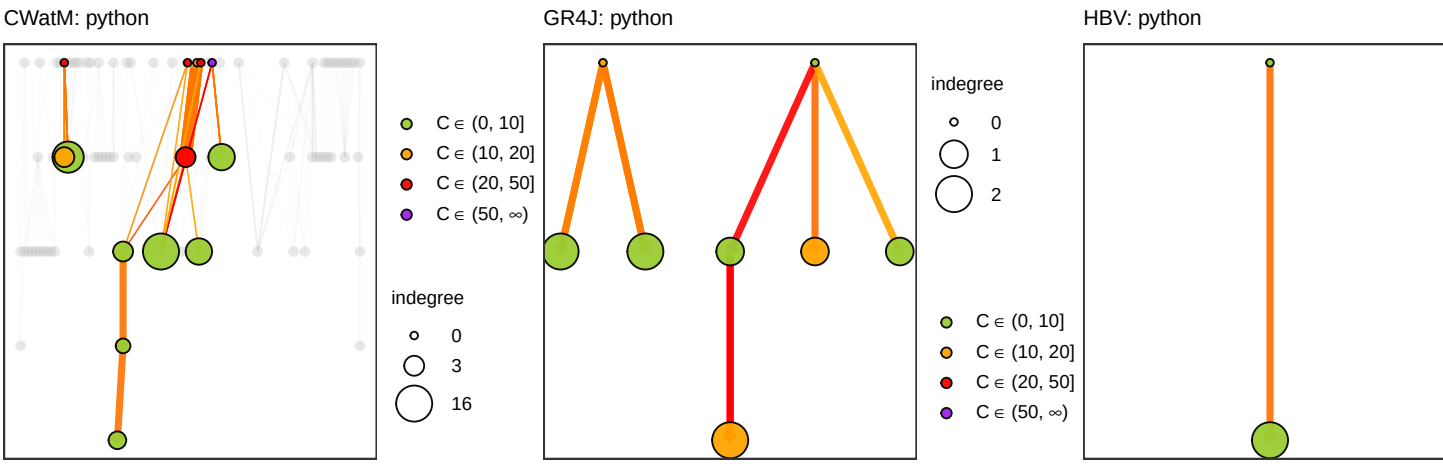

```
# PLOT FIGURES ################################################################

# Define language of models ---------------------------------------------------

python_models <- c("CWatM", "GR4J", "HBV", "PCR-GLOBWB")
fortran_models <- c("ORCHIDEE", "H08", "HYPE", "DBH", "SACRAMENTO")
python_and_fortran <- c("CTSM", "SWAT", "MHM", "HydroPy", "VIC")

all_graphs[, language := fcase(model %chin% python_models, "python",
                               model %chin% fortran_models, "fortran",
                               model %chin% python_and_fortran, "python+fortran",
                               default = NA_character_)]


# Define specifications -------------------------------------------------------

plot_specs <- list(additive = list(paths_col = "paths_tbl_additive",
                                   out_col = "plot_obj_additive"),
                   compensatory = list(paths_col = "paths_tbl_compensatory",
                                       out_col = "plot_obj_compensatory"))
```

```r
# Plot ------------------------------------------------------------------------

# Thickness of edge:frequency across top-10 risky paths
# Color of edge: mean risk of paths using that edge

# Plot graphs -----------------------------------------------------------------

# Override plot_top_paths_fun locally to fix colour-to-complexity mapping:
# named values + drop = FALSE ensure colours are always bound to the same
# complexity range regardless of which levels are present in a given model.
plot_top_paths_fun <- function(graph,
                               all_paths_out,
                               model.name = "",
                               language = "",
                               top_n = 10,
                               alpha_non_top = 0.05) {

  if (is.list(all_paths_out) && !is.data.frame(all_paths_out)) {
    nodes_tbl <- all_paths_out$nodes %||% all_paths_out$nodes_tbl
    paths_tbl <- all_paths_out$paths %||% all_paths_out$paths_tbl
  } else {
    nodes_tbl <- NULL
    paths_tbl <- all_paths_out
  }

  if (is.null(paths_tbl) || !is.data.frame(paths_tbl) || nrow(paths_tbl) == 0) {
    message("No paths in `paths_tbl`; skipping plot for: ", model.name)
    return(invisible(NULL))
  }

  if (is.null(nodes_tbl) || !is.data.frame(nodes_tbl) || nrow(nodes_tbl) == 0) {
    stop("`all_paths_out` must be the output of all_paths_fun() ",
         "(a list with $nodes and $paths containing node-level metrics).",
         call. = FALSE)
  }

  required_node_cols <- c("name", "indeg", "cyclomatic_complexity", "risk_score", "btw")
  missing_node_cols <- setdiff(required_node_cols, names(nodes_tbl))
  if (length(missing_node_cols) > 0) {
    stop("`all_paths_out$nodes` is missing required columns: ",
         paste(missing_node_cols, collapse = ", "), call. = FALSE)
  }

  k <- min(top_n, nrow(paths_tbl))
  top_k_paths <- paths_tbl |>
    dplyr::arrange(dplyr::desc(.data$path_risk_score)) |>
    dplyr::slice_head(n = k)
```

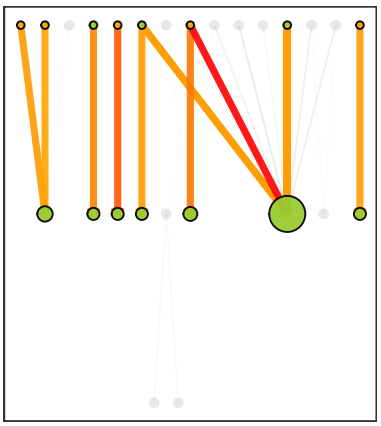
```r
path_edges_all <- purrr::imap_dfr(top_k_paths$path_nodes, function(nodes_vec, pid) {
  tibble::tibble(from = utils::head(nodes_vec, -1), to = utils::tail(nodes_vec, -1),
                 path_id = pid, risk_sum = top_k_paths$risk_sum[pid])
})

ig2 <- tidygraph::as.igraph(graph)
edge_df_names <- igraph::as_data_frame(ig2, what = "edges") |>
  dplyr::mutate(.edge_idx = dplyr::row_number())

path_edges_collapsed <- path_edges_all |>
  dplyr::group_by(.data$from, .data$to) |>
  dplyr::summarise(path_freq = dplyr::n(),
                   risk_mean_path = mean(.data$risk_sum, na.rm = TRUE),
                   .groups = "drop")

edge_marks <- edge_df_names |>
  dplyr::left_join(path_edges_collapsed, by = c("from", "to")) |>
  dplyr::mutate(
    on_top_path    = !is.na(.data$path_freq),
    path_freq      = dplyr::if_else(is.na(.data$path_freq), 0L, as.integer(.data$path_freq)),
    risk_mean_path = dplyr::if_else(is.na(.data$risk_mean_path), 0, .data$risk_mean_path)
  )

graph_sugi <- graph |>
  tidygraph::activate("edges") |>
  dplyr::mutate(on_top_path    = edge_marks$on_top_path,
                path_freq      = edge_marks$path_freq,
                risk_mean_path = edge_marks$risk_mean_path) |>
  tidygraph::activate("nodes") |>
  dplyr::mutate(
    indeg                = nodes_tbl$indeg[match(.data$name, nodes_tbl$name)],
    cyclomatic_complexity = nodes_tbl$cyclomatic_complexity[match(.data$name, nodes_tbl$name)],
    risk_score           = nodes_tbl$risk_score[match(.data$name, nodes_tbl$name)],
    btw                  = nodes_tbl$btw[match(.data$name, nodes_tbl$name)],
    cyclo_class = dplyr::case_when(
      .data$cyclomatic_complexity <= 10 ~ "green",
      .data$cyclomatic_complexity <= 20 ~ "orange",
      .data$cyclomatic_complexity <= 50 ~ "red",
      .data$cyclomatic_complexity >  50 ~ "purple",
      TRUE ~ "grey"
    ),
    complexity_category = cut(
      .data$cyclomatic_complexity,
      breaks = c(-Inf, 10, 20, 50, Inf),
      labels = c("b1", "b2", "b3", "b4")
    )
  )
```

```
risky_nodes <- unique(unlist(top_k_paths$path_nodes))
graph_sugi  <- graph_sugi |>
  tidygraph::activate("nodes") |>
  dplyr::mutate(on_top_node = .data$name %in% risky_nodes)

node_df     <- graph_sugi |> tidygraph::activate("nodes") |> tibble::as_tibble()
risky_indeg <- node_df$indeg[node_df$on_top_node & is.finite(node_df$indeg)]
risky_indeg <- sort(unique(risky_indeg))
if (length(risky_indeg) == 0)
  risky_indeg <- sort(unique(node_df$indeg[is.finite(node_df$indeg)]))

if (length(risky_indeg) >= 3) {
  min_indeg     <- min(risky_indeg)
  max_indeg     <- max(risky_indeg)
  mid_indeg     <- risky_indeg[ceiling(length(risky_indeg) / 2)]
  legend_breaks <- c(min_indeg, mid_indeg, max_indeg)
} else {
  legend_breaks <- risky_indeg
}
legend_labels <- round(legend_breaks, 1)

p_sugi <- ggraph::ggraph(graph_sugi, layout = "sugiyama") +
  ggraph::geom_edge_link0(ggplot2::aes(filter = !.data$on_top_path),
                          colour = "grey80", alpha = alpha_non_top, linewidth = 0.3) +
  ggraph::geom_edge_link0(
    ggplot2::aes(filter = .data$on_top_path, colour = .data$risk_mean_path,
                 linewidth = pmin(pmax(.data$path_freq, 0.5), 3)),
    alpha = 0.9, arrow = grid::arrow(length = grid::unit(1, "mm"))) +
  ggraph::scale_edge_colour_gradient(low = "orange", high = "red", guide = "none") +
  ggraph::scale_edge_width(range = c(0.3, 2.2), guide = "none") +
  ggraph::geom_node_point(size = 1.2, colour = "#BDBDBD", alpha = 0.35, show.legend = FALSE)
  ggraph::geom_node_point(
    ggplot2::aes(filter = .data$on_top_node, size = .data$indeg, fill = .data$complexity_cat
    shape = 21, alpha = 0.95, show.legend = TRUE) +
  ggplot2::scale_size_continuous(breaks = legend_breaks, labels = legend_labels, name = "inde
  ggplot2::scale_fill_manual(
    values = c(b1 = "yellowgreen", b2 = "orange", b3 = "red", b4 = "purple"),
    labels = c(b1 = expression(C %in% "(" * 0 * ", 10" * "]"),
               b2 = expression(C %in% "(" * 10 * ", 20" * "]"),
               b3 = expression(C %in% "(" * 20 * ", 50" * "]"),
               b4 = expression(C %in% "(" * 50 * ", " * infinity * ")")),
    drop = FALSE,
    name = ""
  ) +
  theme_AP() +
  ggplot2::labs(x = "", y = "") +
  ggplot2::theme(
```

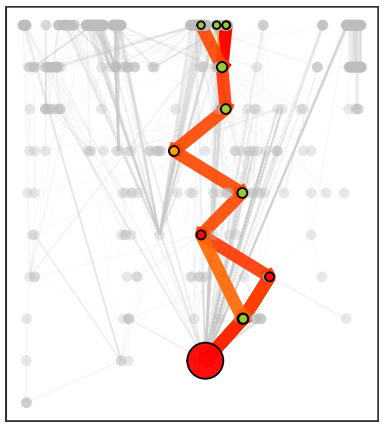
```
      axis.text.y  = ggplot2::element_blank(),
      axis.ticks.y = ggplot2::element_blank(),
      axis.text.x  = ggplot2::element_blank(),
      axis.ticks.x = ggplot2::element_blank(),
      legend.position = "right",
      plot.margin  = ggplot2::margin(1, 1, 1, 1),
      panel.spacing = grid::unit(0.1, "lines")
    ) +
    ggplot2::ggtitle(paste(model.name, ": ", language, sep = ""))

  print(p_sugi)
  invisible(p_sugi)
}

set.seed(seed)

for (nm in names(plot_specs)) {
  spec <- plot_specs[[nm]]

  all_graphs[, (spec$out_col):= mapply(FUN = plot_top_paths_fun, graph = graph,
                                       all_paths_out= get(spec$paths_col),
                                       model.name = model, language = language,
                                       SIMPLIFY = FALSE)]
}
```



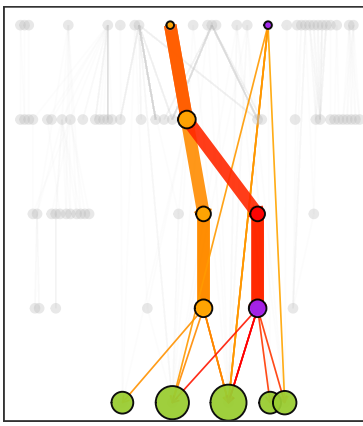
HydroPy: python+fortran



MHM: python+fortran

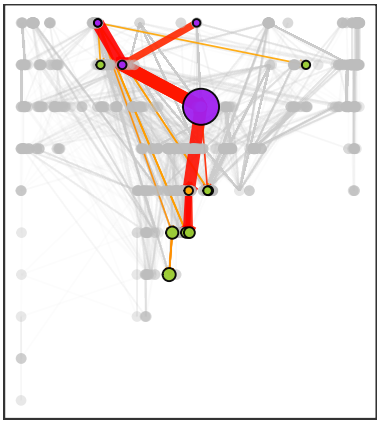


PCR−GLOBWB: python

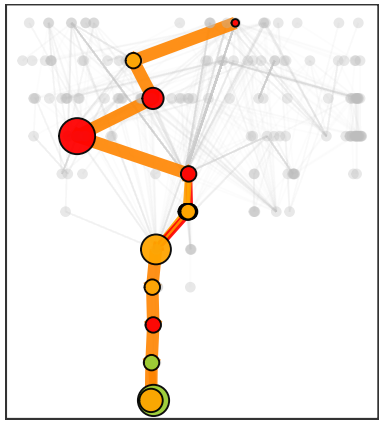


```
## No paths in `paths_tbl`; skipping plot for: VIC
```

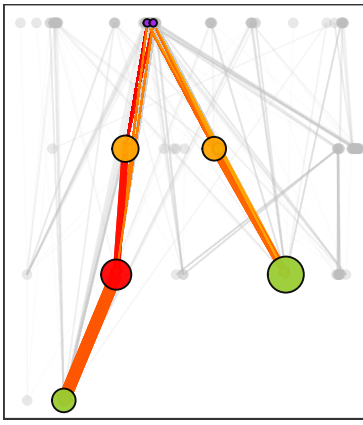
CTSM: python+fortran



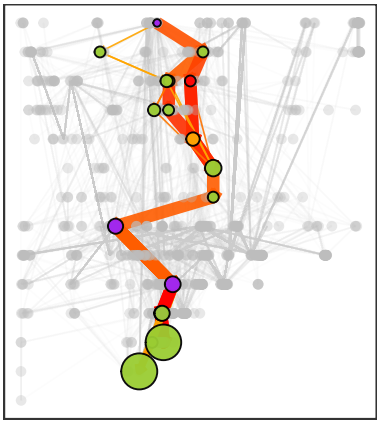
DBH: fortran



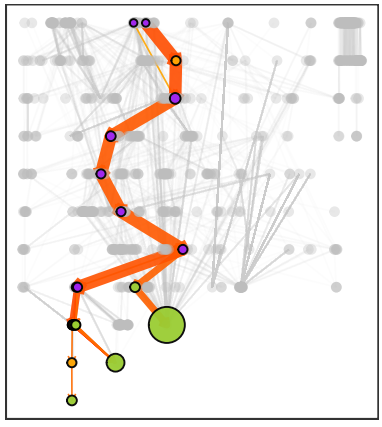
H08: fortran



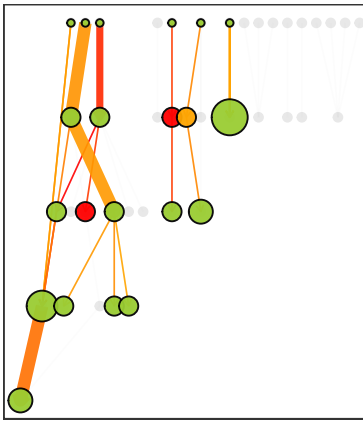
HYPE: fortran

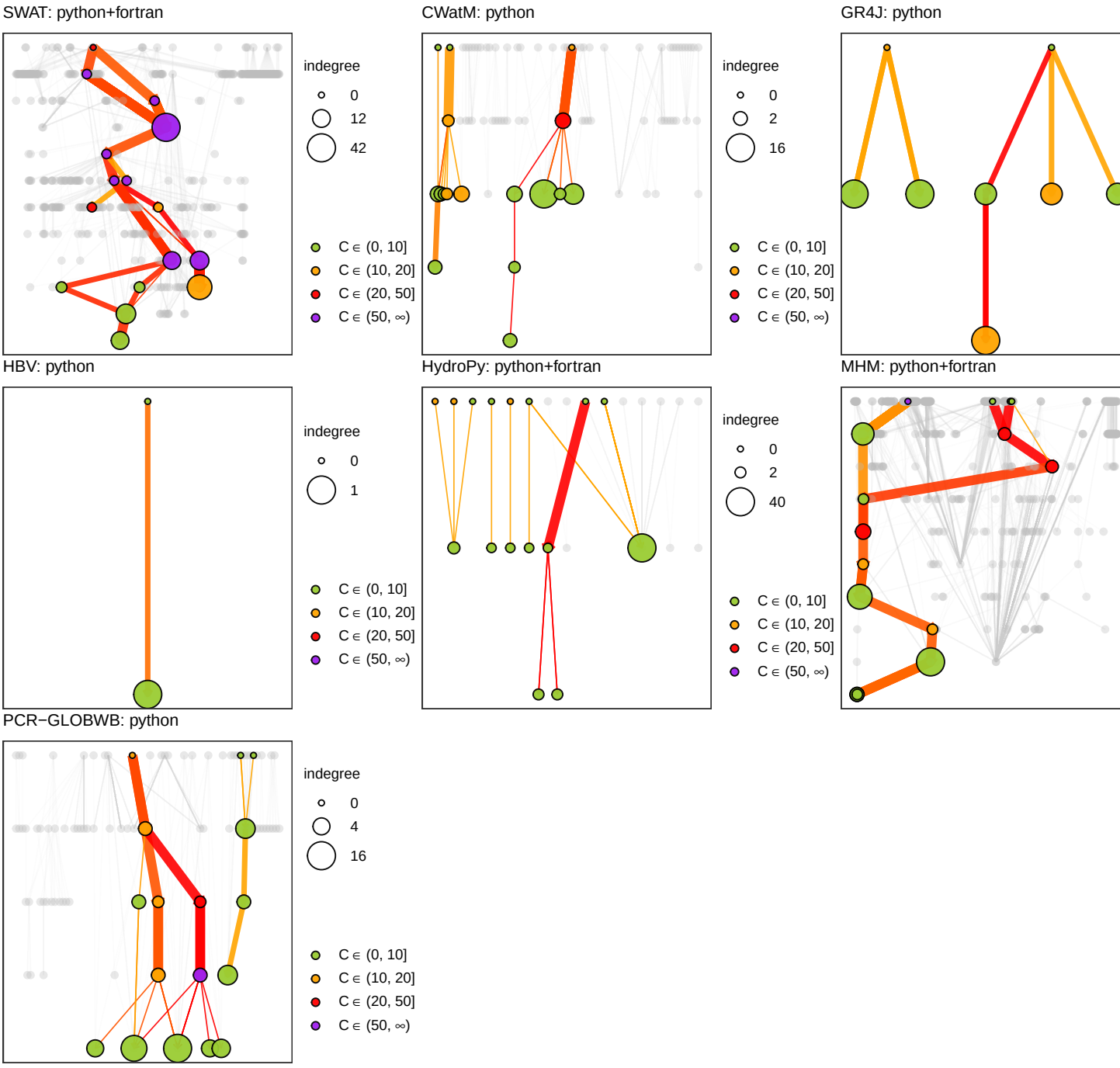


ORCHIDEE: fortran

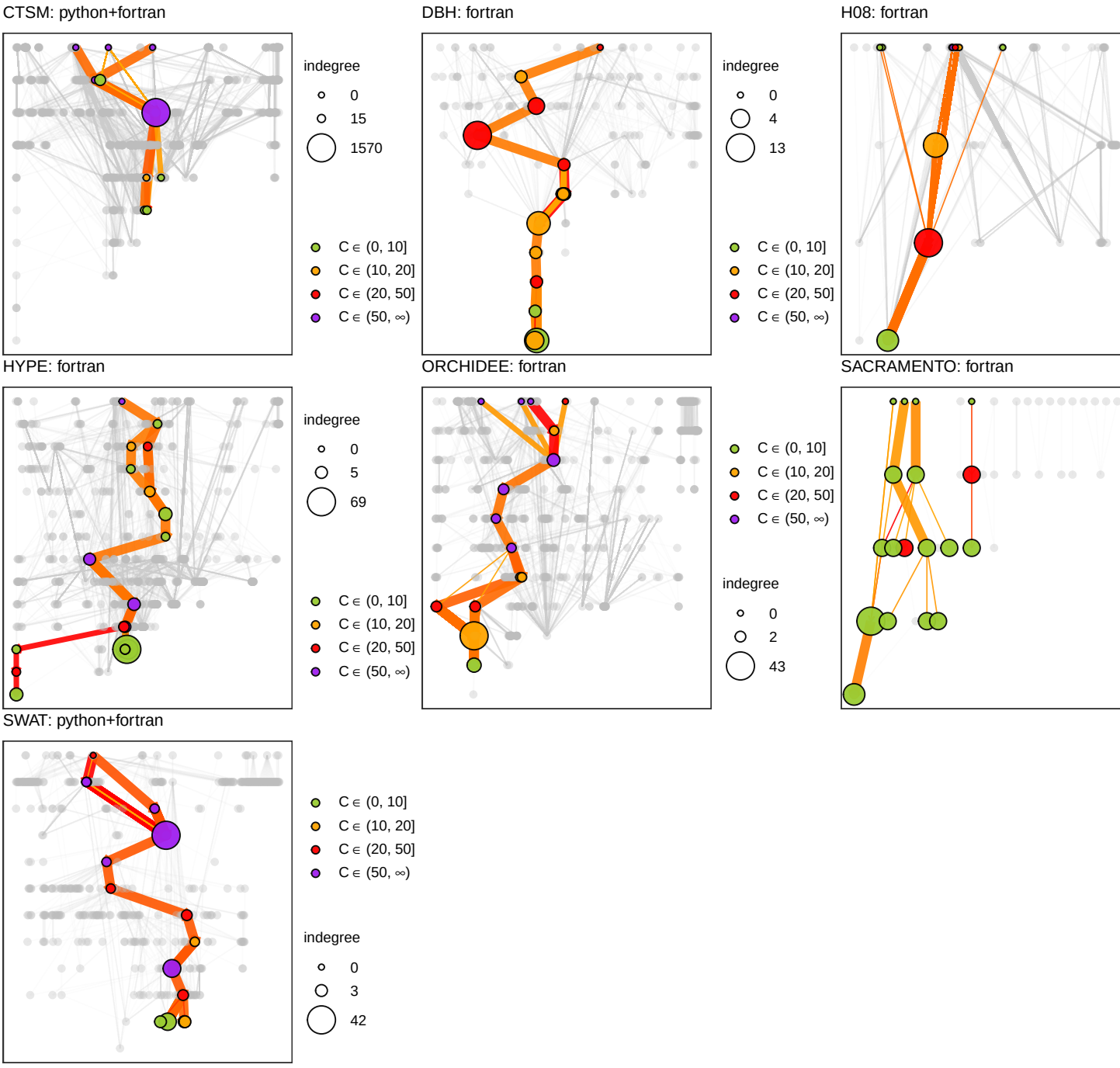


SACRAMENTO: fortran






```
## No paths in `paths_tbl`; skipping plot for: VIC
```




```
# Remove the fill legend ------------------------------------------------------

remove_fill_legend <- function(p) {
  p + guides(fill = "none")
}

plot_cols <- grep("^plot_obj_", names(all_graphs), value = TRUE)

for (col in plot_cols) {
```

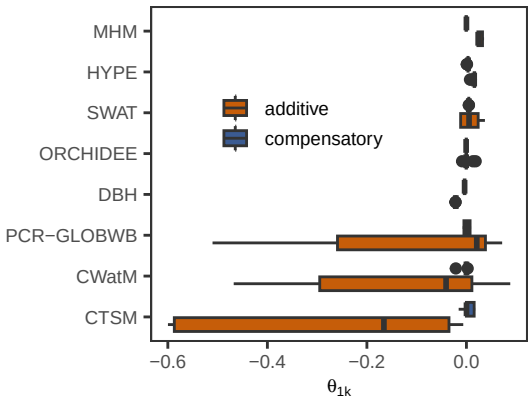
```
  all_graphs[, (col) := lapply(get(col), remove_fill_legend)]}

# Define models to plot ------------------------------------------------------

selected_models <- data.table(model = c("CTSM", "PCR-GLOBWB", "DBH", "HYPE",
                                        "ORCHIDEE", "SWAT", "CWatM", "MHM"))

# Plot call graphs -----------------------------------------------------------

tmp <- all_graphs[selected_models, on = .(model)]
plot_all_risky_paths <- plot_grid(plotlist = tmp$plot_obj_additive, ncol = 2, align = "hv")
plot_all_risky_paths <- plot_grid(legend2, plot_all_risky_paths, ncol = 1, rel_heights = c(0.1
```

```
# HOW MANY OF THE TOP N PATHS ARE THE SAME IN THE ADDITIVE AND COMPENSATORY? ###

top_n <- 10
top10 <- paths_us_all[, .SD[order(-P_median)][1:top_n], .(model, risk_form)]

# Separate the two risk definitions ------------------------------------------

top_add <- unique(top10[risk_form == "additive",      .(model, path_id)])
top_com <- unique(top10[risk_form == "compensatory",  .(model, path_id)])

# Count overlaps per model ---------------------------------------------------

top_add[top_com, on = .(model, path_id), .N, model] %>%
  na.omit() %>%
  .[selected_models, on = .(model)] %>%
  .[order(-N)]
```

```
##         model     N
##        <char> <int>
## 1:       CTSM    10
## 2: PCR-GLOBWB    10
## 3:        DBH    10
## 4:       HYPE    10
## 5:   ORCHIDEE    10
## 6:       SWAT    10
## 7:      CWatM    10
## 8:        MHM    10
```

```
# PLOT DISTRIBUTION OF RISK SLOPE AND GINI FOR THE TOP TEN PATHS ###############

plot_spread_risk <- paths_all[order(-path_risk_score), .SD[1:10], .(model, risk_form)] %>%
  .[selected_models, on = .(model)] %>%
  .[risk_form %in% c("additive", "compensatory")] %>%
  ggplot(., aes(reorder(model, risk_slope), risk_slope, fill = risk_form)) +
  scale_fill_manual(values = color_functional_forms, name = "") +
```

```
  geom_boxplot() +
  labs(x = "", y = expression(theta[1*k])) +
  coord_flip() +
  theme_AP() +
  theme(legend.position = c(0.4, 0.7))

plot_spread_risk
```



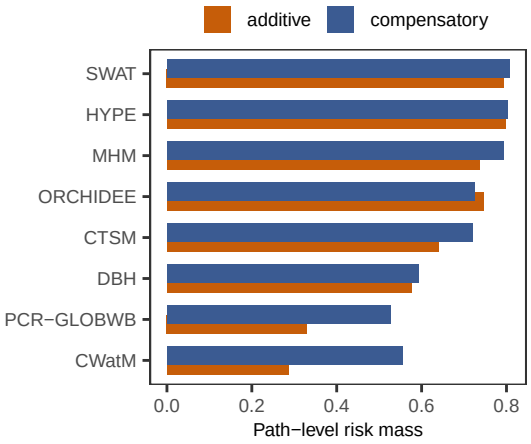

```
# PATH-LEVEL RISK ACCOUNTED FOR THE TOP 5% NODES ###################################

# To long format ---------------------------------------------------------------

paths_long <- paths_all[, .(node = unlist(path_nodes),
                            path_risk_score = path_risk_score,
                            gini_node_risk = gini_node_risk,
                            risk_slope = risk_slope,
                            risk_mean = risk_mean,
                            risk_sum  = risk_sum),
                        .(model, path_id, risk_form)]

# Aggregate at function level --------------------------------------------------

node_from_paths <- paths_long[, .(n_paths = .N,
                                  mean_p_path = mean(path_risk_score, na.rm = TRUE),
                                  max_p_path = max(path_risk_score,  na.rm = TRUE),
                                  sum_p_path = sum(path_risk_score,  na.rm = TRUE),
                                  mean_gini = mean(gini_node_risk, na.rm = TRUE),
                                  mean_slope = mean(risk_slope, na.rm = TRUE),
                                  mean_risksum = mean(path_risk_score, na.rm = TRUE)),
                              .(model, node, risk_form)]

# Join with nodes --------------------------------------------------------------

node_summary <- merge(node_from_paths, nodes_all[, .(name, model, risk_score, risk_form)],
                      by.x = c("model", "node", "risk_form"),
                      by.y = c("model", "name", "risk_form"), all.x = TRUE)
```

```r
# Calculate risk mass ------------------------------------------------------

node_summary[, risk_mass:= mean_p_path * n_paths]

# share of risk mass in top X% nodes, per model .-------------------------------

top_share <- function(X = 0.05) {

  node_summary[!is.na(risk_mass) & risk_mass >= 0, {
    dt <- .SD[order(-risk_mass)]
    n_top <- max(1L, ceiling(.N * X))
    .(X = X, n_nodes = .N, n_top = n_top,
      share_risk_mass_topX = sum(dt$risk_mass[1:n_top]) / sum(dt$risk_mass))
  },
  .(model, risk_form)
  ]
}

# Run function ------------------------------------------------------------------

tmp <- top_share(0.05)

table_top5 <- tmp %>%
  dcast(., model ~ risk_form, value.var = "share_risk_mass_topX") %>%
  .[order(-additive)]

table_top5
```

```
##          model  additive compensatory
##         <char>     <num>        <num>
##  1:       HYPE 0.7985366    0.8016064
##  2:       SWAT 0.7942209    0.8065609
##  3:   ORCHIDEE 0.7460796    0.7251192
##  4:        MHM 0.7355383    0.7931651
##  5:       CTSM 0.6388660    0.7203974
##  6:        DBH 0.5763146    0.5924118
##  7:        HBV 0.5000000    0.5000000
##  8:        H08 0.4726677    0.6384604
##  9: PCR-GLOBWB 0.3300054    0.5273031
## 10:      CWatM 0.2862374    0.5544199
## 11:       GR4J 0.2726486    0.3333333
## 12:    HydroPy 0.2706942    0.6666667
## 13: SACRAMENTO 0.2418015    0.3183764
```

```r
table_top5 %>%
  .[selected_models, on = .(model)] %>%
  melt(., measure.vars = c("additive", "compensatory")) %>%
```

```
  ggplot(., aes(reorder(model, value), value, fill = variable)) +
  geom_bar(stat = "identity", position = position_dodge(0.5)) +
  coord_flip() +
  labs(y = "Path-level risk mass", x = "") +
  scale_fill_manual(values = color_functional_forms, name = "") +
  theme_AP() +
  theme(legend.position = "top")
```



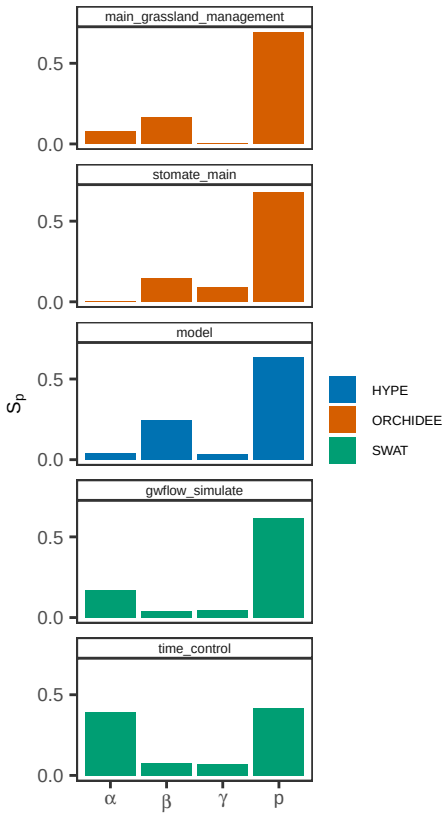

```
# Rerun name of the functions that dominate the top 5% of risk mass -----------

top_share_functions <- function(X = 0.05) {

  node_summary[
    !is.na(risk_mass) & risk_mass >= 0,
    {
      dt <- .SD[order(-risk_mass)]
      n_top <- max(1L, ceiling(.N * X))

      .(
        X = X,
        n_nodes = .N,
        n_top = n_top,
        share_risk_mass_topX = sum(dt$risk_mass[1:n_top]) / sum(dt$risk_mass),
        top_nodes = list(dt$node[1:n_top])
      )
    },
    by = .(model, risk_form)
  ]
}

tmp <- top_share_functions(0.05)
tmp
```

```
##          model    risk_form     X n_nodes n_top share_risk_mass_topX
##         <char>       <char> <num>   <int> <num>                <num>
```

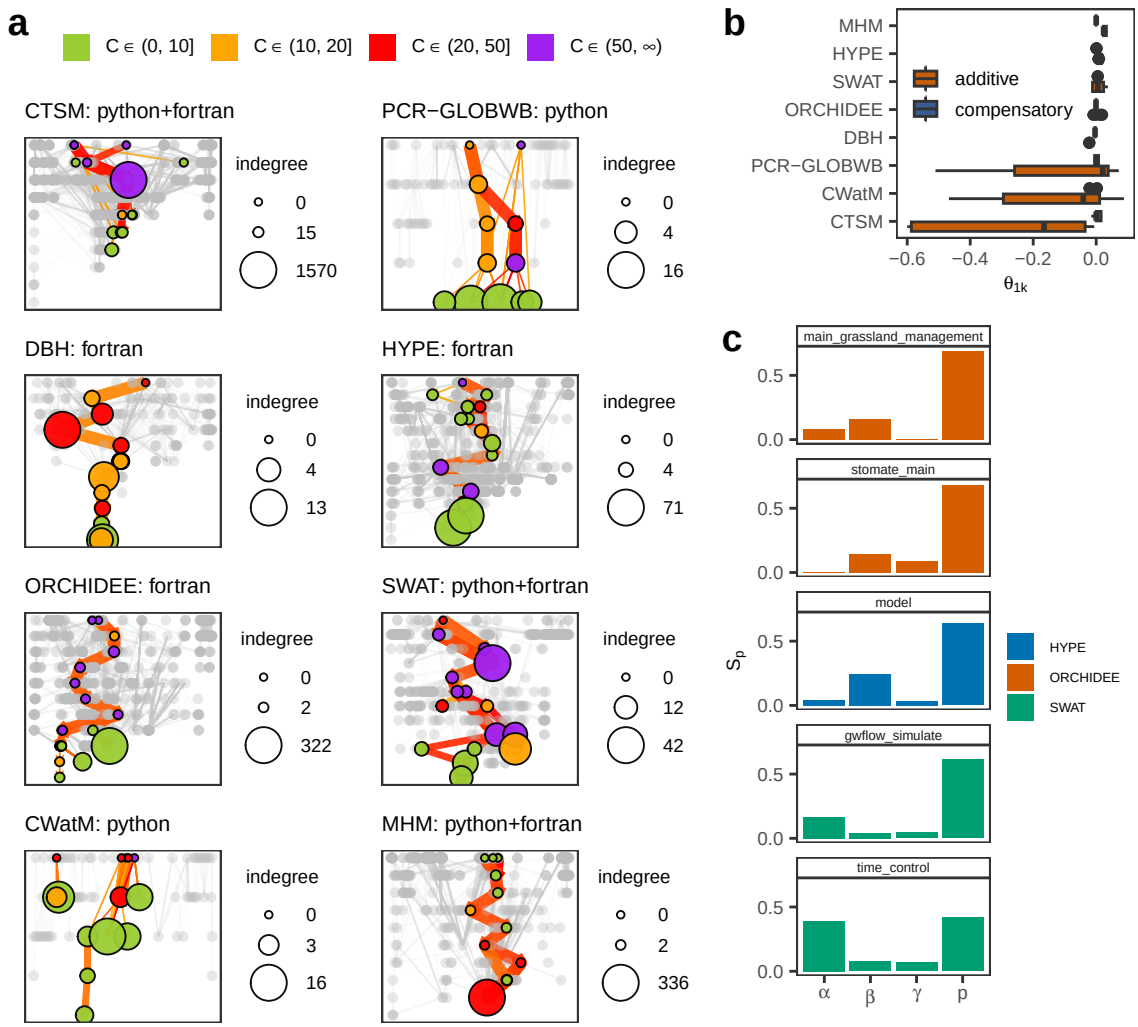
```
##  1:       CTSM     additive  0.05    1124    57            0.6388660
##  2:       CTSM compensatory  0.05    1124    57            0.7203974
##  3:      CWatM     additive  0.05      84     5            0.2862374
##  4:      CWatM compensatory  0.05      84     5            0.5544199
##  5:        DBH     additive  0.05     191    10            0.5763146
##  6:        DBH compensatory  0.05     191    10            0.5924118
##  7:       GR4J     additive  0.05       8     1            0.2726486
##  8:       GR4J compensatory  0.05       8     1            0.3333333
##  9:        H08     additive  0.05     129     7            0.4726677
## 10:        H08 compensatory  0.05     129     7            0.6384604
## 11:        HBV     additive  0.05       2     1            0.5000000
## 12:        HBV compensatory  0.05       2     1            0.5000000
## 13:       HYPE     additive  0.05     872    44            0.7985366
## 14:       HYPE compensatory  0.05     872    44            0.8016064
## 15:    HydroPy     additive  0.05      26     2            0.2706942
## 16:    HydroPy compensatory  0.05      26     2            0.6666667
## 17:        MHM     additive  0.05     565    29            0.7355383
## 18:        MHM compensatory  0.05     565    29            0.7931651
## 19:   ORCHIDEE     additive  0.05     865    44            0.7460796
## 20:   ORCHIDEE compensatory  0.05     865    44            0.7251192
## 21: PCR-GLOBWB     additive  0.05      89     5            0.3300054
## 22: PCR-GLOBWB compensatory  0.05      89     5            0.5273031
## 23: SACRAMENTO     additive  0.05      41     3            0.2418015
## 24: SACRAMENTO compensatory  0.05      41     3            0.3183764
## 25:       SWAT     additive  0.05     481    25            0.7942209
## 26:       SWAT compensatory  0.05     481    25            0.8065609
##          model    risk_form     X n_nodes n_top share_risk_mass_topX
##         <char>       <char> <num>   <int> <num>                <num>
##
##
##  1:                                clm_drv,initialize2,ModelAdvance,lnd_run,InitializeReali
##  2:           initialize2,clm_drv,hist_addfld1d,get_proc_bounds,allhistfldlist_addfld,Initia
##  3:                                                 cbinding,loadsetclone,run_from_command_li
##  4:                                               loadsetclone,run_from_command_line,main,lo
##  5:                             Get_Grid_ATM,Read_Interpolate_data,LOTPS,(main:ETPTREND.f)
##  6:                             Get_Grid_ATM,Read_Interpolate_data,LOTPS,LOCAL,(main:ETPTRE
##  7:
##  8:
##  9:                                read_binary,main_human,main_ftcs,wrte_binary,read_result
## 10:                                wrte_binary,wrte_bints2,read_binary,read_result,main_hum
## 11:
## 12:
## 13:           HYSS,model,function_to_minim,run_model_crit,linesearch_methods_calibration,run
## 14:           HYSS,model,function_to_minim,run_model_crit,linesearch_methods_calibration,run
## 15:                                                                          log_cells.add_va
## 16:                                                                             routing_casca
## 17:                          message,mhm_interface_init,mhm_eval,mhm_interface_run_prepare,m
## 18:                    mhm_interface_init,caldat,dec2date,get_time_vector_and_select,read_ne
```

```
## 19:                                  sechiba_main,slowproc_main,stomate_main,StomateLpj,ipsl
## 20:                       sechiba_main,slowproc_main,stomate_main,StomateLpj,sechiba_initializ
## 21:                        readPCRmapClone,singleTryReadPCRmapClone,getMapAttributesALL,main,
## 22: readPCRmapClone,singleTryReadPCRmapClone,netcdf2PCRobjClone,singleTryNetcdf2PCRobjClone
## 23:                                                                initialize_from_file,sac_initial
## 24:                                                                  initialize_from_file,sac_init
## 25:                               time_control,calsoft_control,command,calsoft_hyd_bfr,actions
## 26:                               time_control,calsoft_control,command,actions,calsoft_hyd_bfr
##
##
```

```
# SENSITIVITY OF SELECTED FUNCTIONS ##############################################

functions_to_check <- c("main_grassland_management", "stomate_main", "model",
                        "gwflow_simulate", "time_control")

plot_sa_functions <- indices_all[name %in% functions_to_check] %>%
  .[, name:= factor(name, levels = functions_to_check)] %>%
  .[sensitivity == "Si"] %>%
  .[risk_form == "additive"] %>%
  ggplot(., aes(parameters, original, fill = model)) +
  geom_bar(stat = "identity") +
  labs(x = "", y = expression(S[p])) +
  scale_fill_manual(values = c("#0072B2", "#D55E00", "#009E73"), name = "") +
  scale_x_discrete(labels = c(alpha = expression(alpha),
                              beta  = expression(beta),
                              gamma = expression(gamma),
                              p     = expression(p))) +
  facet_wrap(~name, ncol = 1) +
  theme_AP() +
  scale_y_continuous(breaks = breaks_pretty(n = 2)) +
  theme(legend.position = "right",
        legend.text = element_text(size = 5),
        strip.text.y.right = element_text(size = 5),
        strip.text.x.top = element_text(size = 4.5),
        strip.text.y = element_text(size = 7.4),
        strip.text = element_text(margin = margin(t = 1.5, b = 1.5)))

plot_sa_functions
```



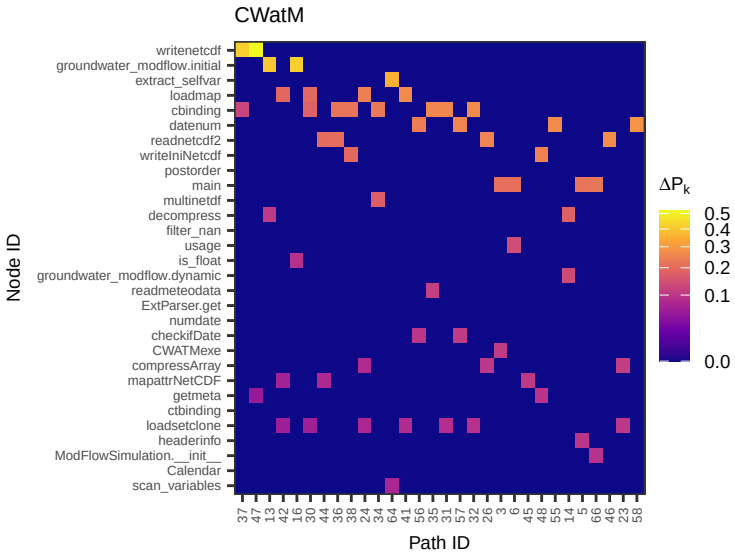

```
# MERGE FIGURES ###################################################################

top <- plot_grid(plot_spread_risk, plot_sa_functions, ncol = 1,
                 rel_heights = c(0.3, 0.7), labels = c("b", "c"))

plot_grid(plot_all_risky_paths, top, rel_widths = c(0.62, 0.38),
          labels = c("a", ""))
```




```
# WHICH PATHS IMPROVE IF A NODE'S RISK IS FIXED TO 0? ############################

number_of_interest <- 30

all_graphs[!model == "VIC",
           path_fix_heatmap:= lapply(paths_tbl_additive,
                                     function(x) {
                                       path_fix_heatmap(all_paths_out = x,
                                                        n_nodes = number_of_interest,
                                                        k_paths = number_of_interest)
  }
)]

# Add name of the model by reference ------------------------------------------

all_graphs[, path_fix_heatmap:= Map(
  function(res, m) {if (is.null(res) || is.null(res$plot)) return(res)
    res$plot <- res$plot + ggtitle(m)
    res
```

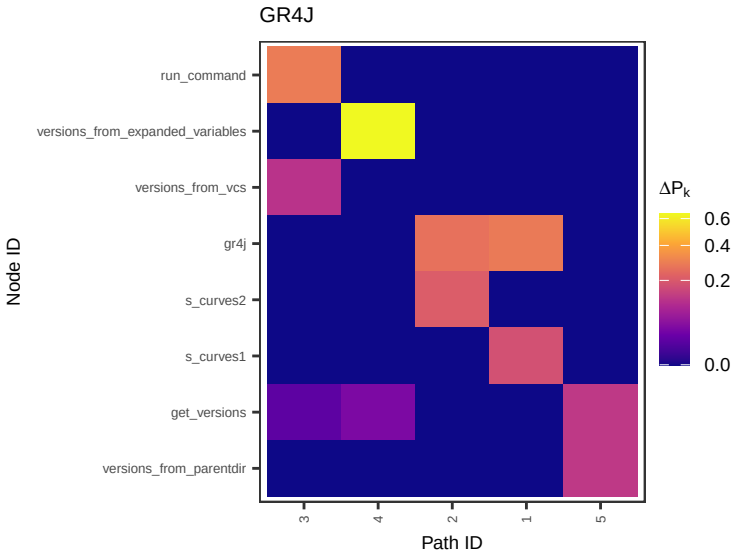
```
  },
  path_fix_heatmap,
  model
)]

# Plot ------------------------------------------------------------------------

all_graphs$path_fix_heatmap
```

```
## [[1]]
## [[1]]$delta_tbl
## # A tibble: 900 x 3
##    node        path_id deltaR
##    <fct>       <fct>    <dbl>
##  1 writenetcdf 37       0.416
##  2 writenetcdf 47       0.521
##  3 writenetcdf 13       0
##  4 writenetcdf 42       0
##  5 writenetcdf 16       0
##  6 writenetcdf 30       0
##  7 writenetcdf 44       0
##  8 writenetcdf 36       0
##  9 writenetcdf 38       0
## 10 writenetcdf 24       0
## # i 890 more rows
##
## [[1]]$plot
```

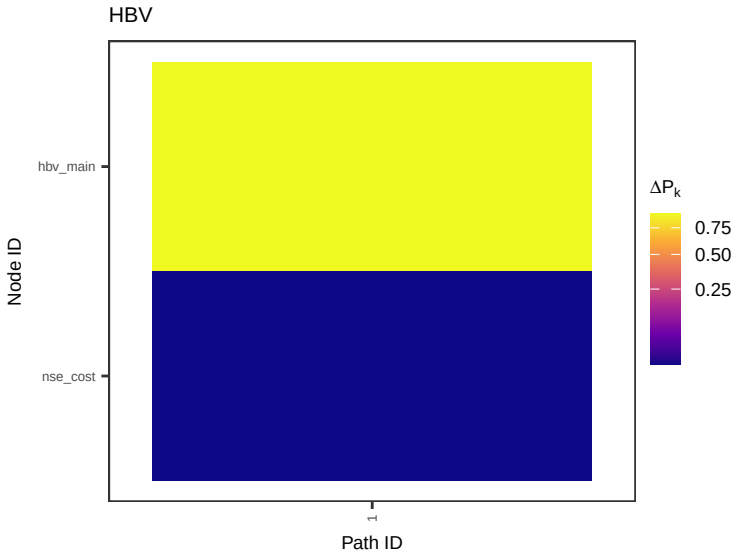



```
##
##
## [[2]]
## [[2]]$delta_tbl
```

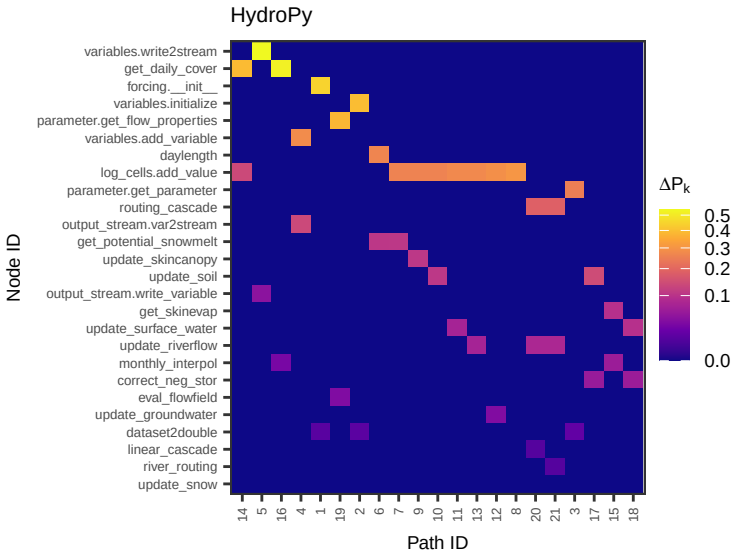
```
## # A tibble: 40 x 3
##    node                                path_id deltaR
##    <fct>                               <fct>    <dbl>
##  1 run_command                         3        0.287
##  2 run_command                         4        0
##  3 run_command                         2        0
##  4 run_command                         1        0
##  5 run_command                         5        0
##  6 versions_from_expanded_variables 3        0
##  7 versions_from_expanded_variables 4        0.638
##  8 versions_from_expanded_variables 2        0
##  9 versions_from_expanded_variables 1        0
## 10 versions_from_expanded_variables 5        0
## # i 30 more rows
##
## [[2]]$plot
```



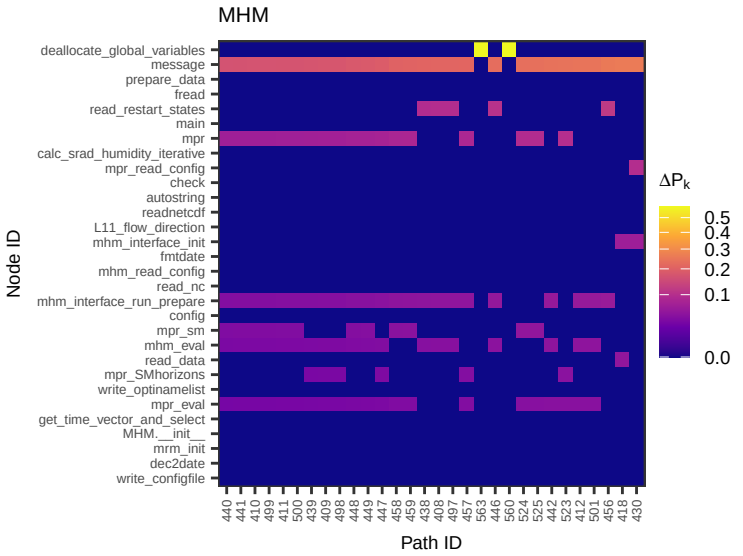

```
##
##
## [[3]]
## [[3]]$delta_tbl
## # A tibble: 2 x 3
##   node     path_id  deltaR
##   <fct>    <fct>     <dbl>
## 1 hbv_main 1       0.902
## 2 nse_cost 1       0.00250
##
## [[3]]$plot
```



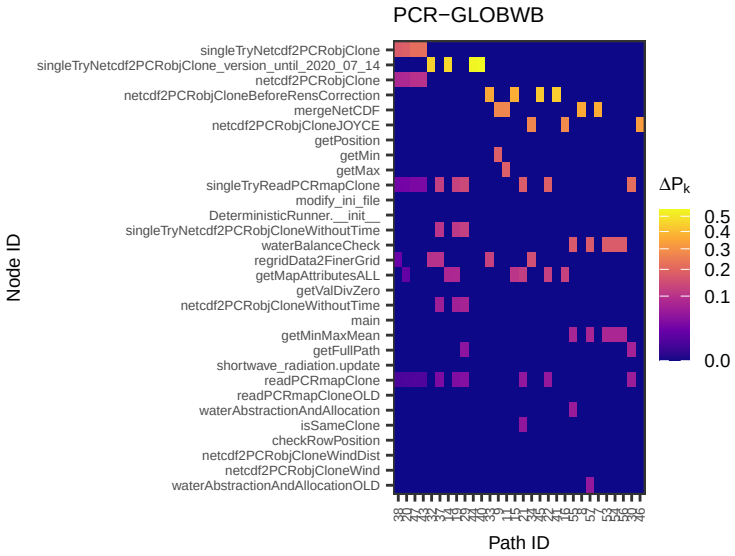

```
##
##
## [[4]]
## [[4]]$delta_tbl
## # A tibble: 546 x 3
##    node                   path_id deltaR
##    <fct>                  <fct>    <dbl>
##  1 variables.write2stream 14       0
##  2 variables.write2stream 5        0.536
##  3 variables.write2stream 16       0
##  4 variables.write2stream 4        0
##  5 variables.write2stream 1        0
##  6 variables.write2stream 19       0
##  7 variables.write2stream 2        0
##  8 variables.write2stream 6        0
##  9 variables.write2stream 7        0
## 10 variables.write2stream 9        0
## # i 536 more rows
##
## [[4]]$plot
```



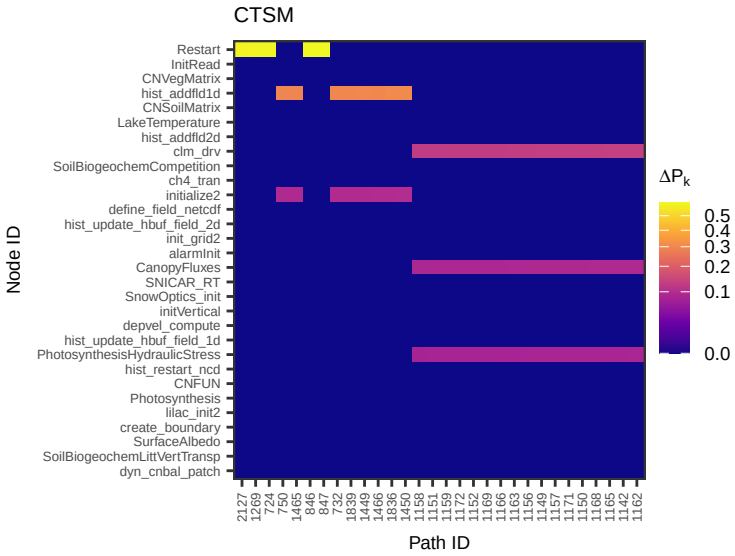

```
##
##
## [[5]]
## [[5]]$delta_tbl
## # A tibble: 900 x 3
##    node                          path_id deltaR
##    <fct>                         <fct>    <dbl>
##  1 deallocate_global_variables 440          0
##  2 deallocate_global_variables 441          0
##  3 deallocate_global_variables 410          0
##  4 deallocate_global_variables 499          0
##  5 deallocate_global_variables 411          0
##  6 deallocate_global_variables 500          0
##  7 deallocate_global_variables 439          0
##  8 deallocate_global_variables 409          0
##  9 deallocate_global_variables 498          0
## 10 deallocate_global_variables 448          0
## # i 890 more rows
##
## [[5]]$plot
```



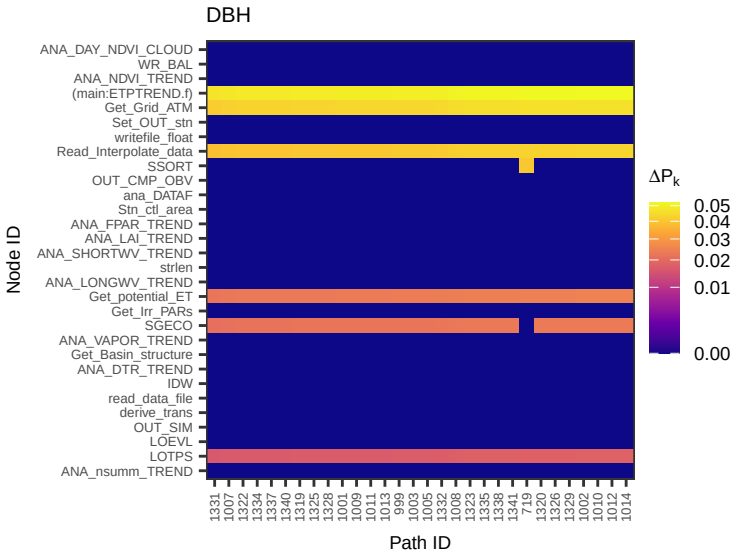

```
##
##
## [[6]]
## [[6]]$delta_tbl
## # A tibble: 900 x 3
##    node                          path_id deltaR
##    <fct>                         <fct>    <dbl>
##  1 singleTryNetcdf2PCRobjClone 38       0.175
##  2 singleTryNetcdf2PCRobjClone 20       0.184
##  3 singleTryNetcdf2PCRobjClone 47       0.213
##  4 singleTryNetcdf2PCRobjClone 43       0.216
##  5 singleTryNetcdf2PCRobjClone 32       0
##  6 singleTryNetcdf2PCRobjClone 37       0
##  7 singleTryNetcdf2PCRobjClone 14       0
##  8 singleTryNetcdf2PCRobjClone 19       0
##  9 singleTryNetcdf2PCRobjClone 29       0
## 10 singleTryNetcdf2PCRobjClone 44       0
## # i 890 more rows
##
## [[6]]$plot
```



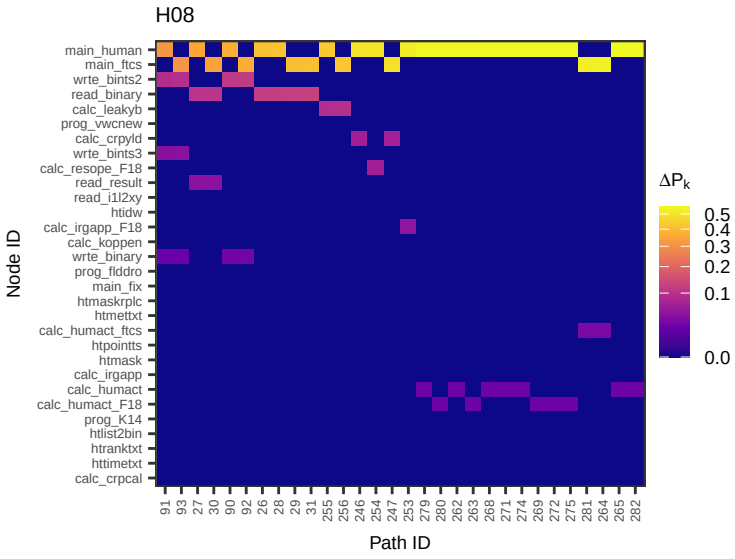

```
##
##
## [[7]]
## NULL
##
## [[8]]
## [[8]]$delta_tbl
## # A tibble: 900 x 3
##    node    path_id deltaR
##    <fct>   <fct>    <dbl>
##  1 Restart 2127     0.579
##  2 Restart 1269     0.591
##  3 Restart 724      0.592
##  4 Restart 750      0
##  5 Restart 1465     0
##  6 Restart 846      0.600
##  7 Restart 847      0.600
##  8 Restart 732      0
##  9 Restart 1839     0
## 10 Restart 1449     0
## # i 890 more rows
##
## [[8]]$plot
```



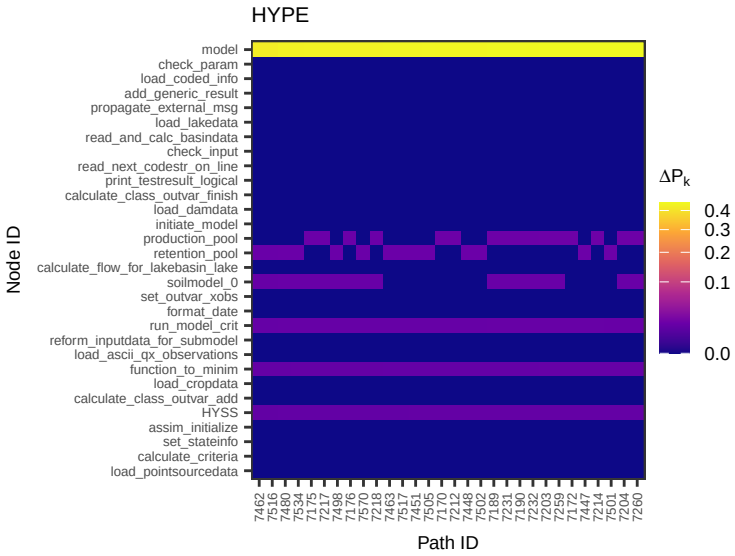

```
##
##
## [[9]]
## [[9]]$delta_tbl
## # A tibble: 900 x 3
##    node                path_id deltaR
##    <fct>               <fct>    <dbl>
##  1 ANA_DAY_NDVI_CLOUD 1331         0
##  2 ANA_DAY_NDVI_CLOUD 1007         0
##  3 ANA_DAY_NDVI_CLOUD 1322         0
##  4 ANA_DAY_NDVI_CLOUD 1334         0
##  5 ANA_DAY_NDVI_CLOUD 1337         0
##  6 ANA_DAY_NDVI_CLOUD 1340         0
##  7 ANA_DAY_NDVI_CLOUD 1319         0
##  8 ANA_DAY_NDVI_CLOUD 1325         0
##  9 ANA_DAY_NDVI_CLOUD 1328         0
## 10 ANA_DAY_NDVI_CLOUD 1001         0
## # i 890 more rows
##
## [[9]]$plot
```



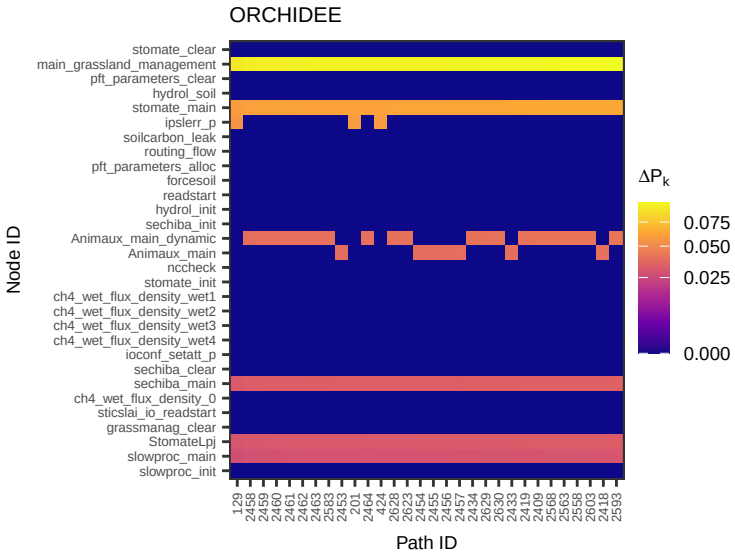

```
##
##
## [[10]]
## [[10]]$delta_tbl
## # A tibble: 900 x 3
##    node       path_id deltaR
##    <fct>      <fct>    <dbl>
##  1 main_human 91       0.314
##  2 main_human 93       0
##  3 main_human 27       0.354
##  4 main_human 30       0
##  5 main_human 90       0.376
##  6 main_human 92       0
##  7 main_human 26       0.413
##  8 main_human 28       0.419
##  9 main_human 29       0
## 10 main_human 31       0
## # i 890 more rows
##
## [[10]]$plot
```



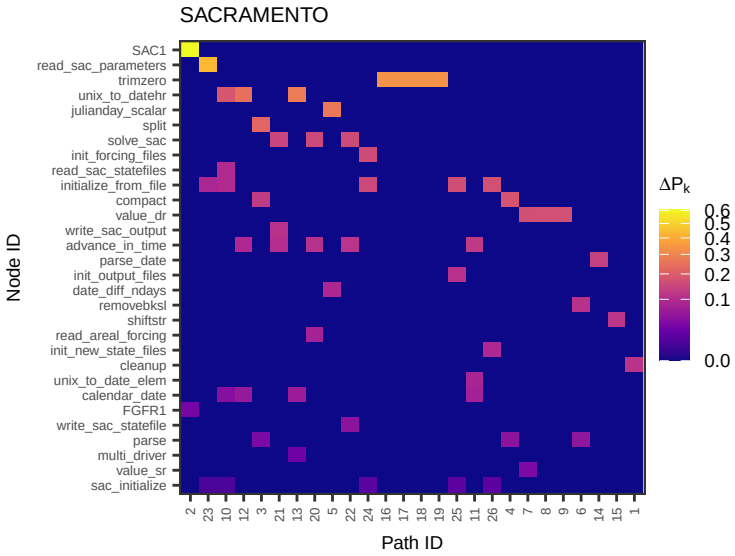

```
##
##
## [[11]]
## [[11]]$delta_tbl
## # A tibble: 900 x 3
##    node  path_id deltaR
##    <fct> <fct>    <dbl>
##  1 model 7462     0.419
##  2 model 7516     0.420
##  3 model 7480     0.430
##  4 model 7534     0.430
##  5 model 7175     0.431
##  6 model 7217     0.432
##  7 model 7498     0.432
##  8 model 7176     0.432
##  9 model 7570     0.432
## 10 model 7218     0.432
## # i 890 more rows
##
## [[11]]$plot
```



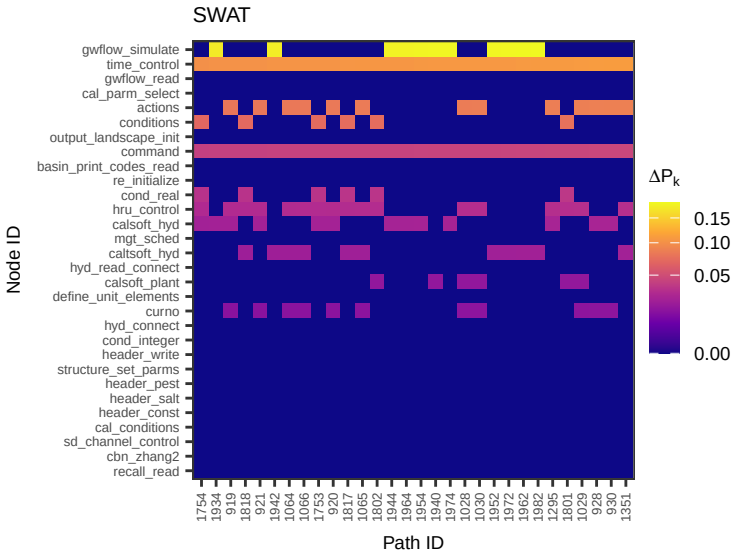

```
##
##
## [[12]]
## [[12]]$delta_tbl
## # A tibble: 900 x 3
##    node          path_id deltaR
##    <fct>         <fct>    <dbl>
##  1 stomate_clear 129          0
##  2 stomate_clear 2458         0
##  3 stomate_clear 2459         0
##  4 stomate_clear 2460         0
##  5 stomate_clear 2461         0
##  6 stomate_clear 2462         0
##  7 stomate_clear 2463         0
##  8 stomate_clear 2583         0
##  9 stomate_clear 2453         0
## 10 stomate_clear 201          0
## # i 890 more rows
##
## [[12]]$plot
```




```
##
##
## [[13]]
## [[13]]$delta_tbl
## # A tibble: 780 x 3
##    node  path_id deltaR
##    <fct> <fct>    <dbl>
##  1 SAC1  2        0.603
##  2 SAC1  23       0
##  3 SAC1  10       0
##  4 SAC1  12       0
##  5 SAC1  3        0
##  6 SAC1  21       0
##  7 SAC1  13       0
##  8 SAC1  20       0
##  9 SAC1  5        0
## 10 SAC1  22       0
## # i 770 more rows
##
## [[13]]$plot
```




```
##
##
## [[14]]
## [[14]]$delta_tbl
## # A tibble: 900 x 3
##    node            path_id deltaR
##    <fct>           <fct>    <dbl>
##  1 gwflow_simulate 1754     0
##  2 gwflow_simulate 1934     0.173
##  3 gwflow_simulate 919      0
##  4 gwflow_simulate 1818     0
##  5 gwflow_simulate 921      0
##  6 gwflow_simulate 1942     0.176
##  7 gwflow_simulate 1064     0
##  8 gwflow_simulate 1066     0
##  9 gwflow_simulate 1753     0
## 10 gwflow_simulate 920      0
## # i 890 more rows
##
## [[14]]$plot
```



3 Real-model scalability

```
# REAL-MODEL SCALABILITY: TIMING AND RANK STABILITY ############################
################################################################################

# This code extracts timing and rank stability metrics from the 14 domain models
# already analyzed here.

set.seed(123)

# 1. RE-TIME THE FULL PIPELINE PER MODEL ########################################
# We re-run all_paths_fun and uncertainty_fun to get clean wall-clock times

N_ua <- 2^10  # same as in the main analysis

real_model_timing <- all_graphs[model != "VIC", {

  g <- graph[[1]]
  m <- model

  cat("\n=== Model:", m, "===\n")

  n_nodes <- igraph::vcount(g)
  n_edges <- igraph::ecount(g)

  # Time all_paths_fun ------------------------------------------------------

  t_paths <- system.time({
    out <- tryCatch(
      softwareRisk::all_paths_fun(
```

```
      graph          = g,
      alpha          = 0.6,
      beta           = 0.3,
      gamma          = 0.1,
      p              = 1,
      complexity_col = "cyclomatic_complexity"
    ),
    error = function(e) {
      cat("  all_paths_fun failed:", e$message, "\n")
      NULL
    }
  )
})["elapsed"]

n_paths <- if (!is.null(out) && nrow(out$paths) > 0) nrow(out$paths) else NA_integer_

cat("  Nodes:", n_nodes, " Edges:", n_edges, " Paths:", n_paths, "\n")

# Time uncertainty_fun (compensatory) ----------------------------------------

t_ua <- NA_real_
if (!is.null(out) && nrow(out$paths) > 0) {
  t_ua <- system.time({
    tryCatch(
      softwareRisk::uncertainty_fun(
        all_paths_out = out,
        N             = N_ua,
        order         = "first"
      ),
      error = function(e) {
        cat("  uncertainty_fun failed:", e$message, "\n")
        NULL
      }
    )
  })["elapsed"]
}

t_total <- t_paths + ifelse(is.na(t_ua), 0, t_ua)

cat("  Time - paths:", round(t_paths, 2),
    "s | UA:", round(t_ua, 2),
    "s | total:", round(t_total, 2), "s\n")

.(n_nodes  = n_nodes,
  n_edges  = n_edges,
  n_paths  = n_paths,
  t_paths  = t_paths,
```

```
    t_ua     = t_ua,
    t_total  = t_total)

}, model]
```

```
##
## === Model: CWatM ===
##   Nodes: 88  Edges: 173  Paths: 79
##   Time - paths: 0.1 s | UA: 2.35 s | total: 2.45 s
##
## === Model: GR4J ===
##   Nodes: 8  Edges: 9  Paths: 5
##   Time - paths: 0.01 s | UA: 0.14 s | total: 0.15 s
##
## === Model: HBV ===
##   Nodes: 2  Edges: 1  Paths: 1
##   Time - paths: 0 s | UA: 0.03 s | total: 0.03 s
##
## === Model: HydroPy ===
##   Nodes: 26  Edges: 56  Paths: 21
##   Time - paths: 0.03 s | UA: 0.62 s | total: 0.65 s
##
## === Model: MHM ===
##   Nodes: 574  Edges: 1268  Paths: 980
##   Time - paths: 1.33 s | UA: 28.28 s | total: 29.61 s
##
## === Model: PCR-GLOBWB ===
##   Nodes: 89  Edges: 239  Paths: 101
##   Time - paths: 0.14 s | UA: 2.95 s | total: 3.09 s
##
## === Model: CTSM ===
##   Nodes: 1124  Edges: 4997  Paths: 2232
##   Time - paths: 3 s | UA: 65.28 s | total: 68.28 s
##
## === Model: DBH ===
##   Nodes: 191  Edges: 867  Paths: 2123
##   Time - paths: 2.18 s | UA: 60.97 s | total: 63.15 s
##
## === Model: H08 ===
##   Nodes: 129  Edges: 1273  Paths: 286
##   Time - paths: 0.38 s | UA: 7.98 s | total: 8.37 s
##
## === Model: HYPE ===
##   Nodes: 872  Edges: 3443  Paths: 10875
##   Time - paths: 11.62 s | UA: 313.44 s | total: 325.06 s
##
## === Model: ORCHIDEE ===
```

```
##   Nodes: 865  Edges: 2538  Paths: 3152
##   Time - paths: 3.98 s | UA: 92.46 s | total: 96.44 s
##
## === Model: SACRAMENTO ===
##   Nodes: 41  Edges: 40  Paths: 26
##   Time - paths: 0.04 s | UA: 0.83 s | total: 0.87 s
##
## === Model: SWAT ===
##   Nodes: 481  Edges: 836  Paths: 4169
##   Time - paths: 4.51 s | UA: 122.13 s | total: 126.64 s
```

```
print(real_model_timing[order(-n_paths)])
```

```
##          model n_nodes n_edges n_paths t_paths    t_ua t_total
##         <char>   <num>   <num>   <int>   <num>   <num>   <num>
##  1:       HYPE     872    3443   10875  11.619 313.438 325.057
##  2:       SWAT     481     836    4169   4.508 122.134 126.642
##  3:   ORCHIDEE     865    2538    3152   3.975  92.463  96.438
##  4:       CTSM    1124    4997    2232   2.998  65.278  68.276
##  5:        DBH     191     867    2123   2.181  60.970  63.151
##  6:        MHM     574    1268     980   1.327  28.283  29.610
##  7:        H08     129    1273     286   0.384   7.984   8.368
##  8: PCR-GLOBWB      89     239     101   0.141   2.948   3.089
##  9:      CWatM      88     173      79   0.104   2.349   2.453
## 10: SACRAMENTO      41      40      26   0.038   0.829   0.867
## 11:    HydroPy      26      56      21   0.033   0.620   0.653
## 12:       GR4J       8       9       5   0.009   0.142   0.151
## 13:        HBV       2       1       1   0.003   0.030   0.033
```

```
# 2. RANK STABILITY FROM EXISTING UA DRAWS ====================================

compute_stability_safe <- function(ua_vec_list, n_p) {

  if (n_p < 10) {
    return(data.table(top1_stability = NA_real_,
                      top10_jaccard = NA_real_,
                      n_paths = n_p))
  }

  n_draws <- length(ua_vec_list[[1]])

  # Build Pk matrix in chunks to avoid single huge allocation
  Pk_matrix <- matrix(NA_real_, nrow = n_p, ncol = n_draws)
  for (i in seq_len(n_p)) {
    Pk_matrix[i, ] <- ua_vec_list[[i]]
  }

  # Rank one column at a time (avoids full rank_matrix in memory)
  top1_counts <- integer(n_p)
```

```
  k_top <- min(10, n_p)

  # Sample pairs for Jaccard upfront
  n_pairs <- min(300, choose(n_draws, 2))
  pi_idx <- sample(n_draws, n_pairs, replace = TRUE)
  pj_idx <- sample(n_draws, n_pairs, replace = TRUE)
  valid <- pi_idx != pj_idx
  pi_idx <- pi_idx[valid]
  pj_idx <- pj_idx[valid]

  # Store top-10 sets per draw (only for sampled draws)
  draws_needed <- unique(c(pi_idx, pj_idx))
  top10_cache <- vector("list", n_draws)

  for (d in seq_len(n_draws)) {
    ranks_d <- frank(-Pk_matrix[, d], ties.method = "average")
    top1_counts[which.min(ranks_d)] <- top1_counts[which.min(ranks_d)] + 1L

    if (d %in% draws_needed) {
      top10_cache[[d]] <- order(ranks_d)[1:k_top]
    }
  }

  top1_stab <- max(top1_counts) / n_draws

  # Jaccard from cached top-10 sets
  jac <- vapply(seq_along(pi_idx), function(idx) {
    a <- top10_cache[[pi_idx[idx]]]
    b <- top10_cache[[pj_idx[idx]]]
    if (is.null(a) || is.null(b)) return(NA_real_)
    length(intersect(a, b)) / length(union(a, b))
  }, numeric(1))

  data.table(top1_stability = top1_stab,
             top10_jaccard = mean(jac, na.rm = TRUE),
             n_paths = n_p)
}

# Process one model-risk_form at a time ----------------------------------------

combos <- unique(paths_us_all[, .(model, risk_form)])

real_model_stability <- rbindlist(lapply(seq_len(nrow(combos)), function(i) {

  m  <- combos$model[i]
  rf <- combos$risk_form[i]
```

```
  cat("  Stability:", m, rf, "\n")

  sub <- paths_us_all[model == m & risk_form == rf]
  result <- compute_stability_safe(sub$uncertainty_analysis, nrow(sub))
  result[, `:=`(model = m, risk_form = rf)]

  gc()  # free memory between models
  result
}))
```

```
##   Stability: CWatM additive
##   Stability: GR4J additive
##   Stability: HBV additive
##   Stability: HydroPy additive
##   Stability: MHM additive
##   Stability: PCR-GLOBWB additive
##   Stability: CTSM additive
##   Stability: DBH additive
##   Stability: H08 additive
##   Stability: HYPE additive
##   Stability: ORCHIDEE additive
##   Stability: SACRAMENTO additive
##   Stability: SWAT additive
##   Stability: CWatM compensatory
##   Stability: GR4J compensatory
##   Stability: HBV compensatory
##   Stability: HydroPy compensatory
##   Stability: MHM compensatory
##   Stability: PCR-GLOBWB compensatory
##   Stability: CTSM compensatory
##   Stability: DBH compensatory
##   Stability: H08 compensatory
##   Stability: HYPE compensatory
##   Stability: ORCHIDEE compensatory
##   Stability: SACRAMENTO compensatory
##   Stability: SWAT compensatory
```

```
print(real_model_stability[order(-n_paths)])
```

```
##     top1_stability top10_jaccard n_paths      model    risk_form
##              <num>         <num>   <int>     <char>       <char>
##  1:      0.4042969     0.3886176   10875       HYPE     additive
##  2:      0.4042969     0.3878392   10875       HYPE compensatory
##  3:      0.1982422     0.1567981    4169       SWAT     additive
##  4:      0.1982422     0.1466385    4169       SWAT compensatory
##  5:      0.3554688     0.2966129    3152   ORCHIDEE     additive
##  6:      0.3554688     0.2852459    3152   ORCHIDEE compensatory
##  7:      0.2724609     0.4214099    2232       CTSM     additive
```

```
##  8:      0.2724609     0.4454147    2232       CTSM compensatory
##  9:      0.5888672     0.4584633    2123        DBH     additive
## 10:      0.5888672     0.4770408    2123        DBH compensatory
## 11:      0.5800781     0.3709377     980        MHM     additive
## 12:      0.5800781     0.3762090     980        MHM compensatory
## 13:      0.3779297     0.4321742     286        H08     additive
## 14:      0.3779297     0.4420474     286        H08 compensatory
## 15:      0.6523438     0.6626481     101 PCR-GLOBWB     additive
## 16:      0.6523438     0.6570132     101 PCR-GLOBWB compensatory
## 17:      0.8271484     0.4451515      79      CWatM     additive
## 18:      0.8271484     0.4455641      79      CWatM compensatory
## 19:      0.8974609     0.7073664      26 SACRAMENTO     additive
## 20:      0.8974609     0.7241630      26 SACRAMENTO compensatory
## 21:      0.6142578     0.4894805      21    HydroPy     additive
## 22:      0.6142578     0.5146668      21    HydroPy compensatory
## 23:             NA            NA       5       GR4J     additive
## 24:             NA            NA       5       GR4J compensatory
## 25:             NA            NA       1        HBV     additive
## 26:             NA            NA       1        HBV compensatory
##     top1_stability top10_jaccard n_paths      model    risk_form
##              <num>         <num>   <int>     <char>       <char>
```

```
# 3. MERGE TIMING AND STABILITY ================================================

real_scalability <- merge(
  real_model_timing,
  real_model_stability[risk_form == "compensatory",
                       .(model, top1_stability, top10_jaccard)],
  by = "model", all.x = TRUE
)

real_scalability <- real_scalability[order(-n_paths)]

cat("\n\n=== REAL MODEL SCALABILITY ===\n")
```

```
##
##
## === REAL MODEL SCALABILITY ===
```

```
print(real_scalability)
```

```
##          model n_nodes n_edges n_paths t_paths    t_ua t_total top1_stability
##         <char>   <num>   <num>   <int>   <num>   <num>   <num>          <num>
##  1:       HYPE     872    3443   10875  11.619 313.438 325.057      0.4042969
##  2:       SWAT     481     836    4169   4.508 122.134 126.642      0.1982422
##  3:   ORCHIDEE     865    2538    3152   3.975  92.463  96.438      0.3554688
##  4:       CTSM    1124    4997    2232   2.998  65.278  68.276      0.2724609
##  5:        DBH     191     867    2123   2.181  60.970  63.151      0.5888672
##  6:        MHM     574    1268     980   1.327  28.283  29.610      0.5800781
```

```
##  7:         H08     129    1273     286   0.384   7.984   8.368      0.3779297
##  8: PCR-GLOBWB      89     239     101   0.141   2.948   3.089      0.6523438
##  9:       CWatM      88     173      79   0.104   2.349   2.453      0.8271484
## 10: SACRAMENTO      41      40      26   0.038   0.829   0.867      0.8974609
## 11:    HydroPy      26      56      21   0.033   0.620   0.653      0.6142578
## 12:       GR4J       8       9       5   0.009   0.142   0.151             NA
## 13:        HBV       2       1       1   0.003   0.030   0.033             NA
##     top10_jaccard
##             <num>
##  1:     0.3878392
##  2:     0.1466385
##  3:     0.2852459
##  4:     0.4454147
##  5:     0.4770408
##  6:     0.3762090
##  7:     0.4420474
##  8:     0.6570132
##  9:     0.4455641
## 10:     0.7241630
## 11:     0.5146668
## 12:            NA
## 13:            NA
```

```
write.xlsx(real_scalability, "real_model_scalability.xlsx")

# 4. PLOTS ========================================================================

# A) Timing by stage per model ------------------------------------------------

time_real_long <- melt(
  real_scalability[!is.na(t_total)],
  id.vars = c("model", "n_nodes", "n_paths"),
  measure.vars = c("t_paths", "t_ua"),
  variable.name = "stage", value.name = "seconds"
)

time_real_long[, stage := fcase(
  stage == "t_paths", "Path enumeration",
  stage == "t_ua",    "Uncertainty / sensitivity analysis"
)]

time_real_long[, model := factor(model,
  levels = real_scalability[order(n_paths), model])]

# B) Stability per model ------------------------------------------------------

stability_real_long <- melt(
  real_scalability[!is.na(top1_stability)],
```

```
  id.vars = c("model", "n_nodes", "n_paths"),
  measure.vars = c("top1_stability", "top10_jaccard"),
  variable.name = "metric", value.name = "value"
)

stability_real_long[, metric := fcase(
  metric == "top1_stability", "Top-1 stability",
  metric == "top10_jaccard",  "Top-10 Jaccard similarity"
)]

stability_real_long[, model := factor(model,
  levels = real_scalability[order(n_paths), model])]

# OMNIBUS SUMMARY ##################################################################

cat("\n--- Real model timing summary ---\n")
```

```
##
## --- Real model timing summary ---
```

```
print(real_scalability[!is.na(t_total), .(
  median_total_s = round(median(t_total), 1),
  max_total_s    = round(max(t_total), 1),
  median_paths   = median(n_paths),
  max_paths      = max(n_paths)
)])
```

```
##    median_total_s max_total_s median_paths max_paths
##             <num>       <num>        <int>     <int>
## 1:            8.4       325.1          286     10875
```

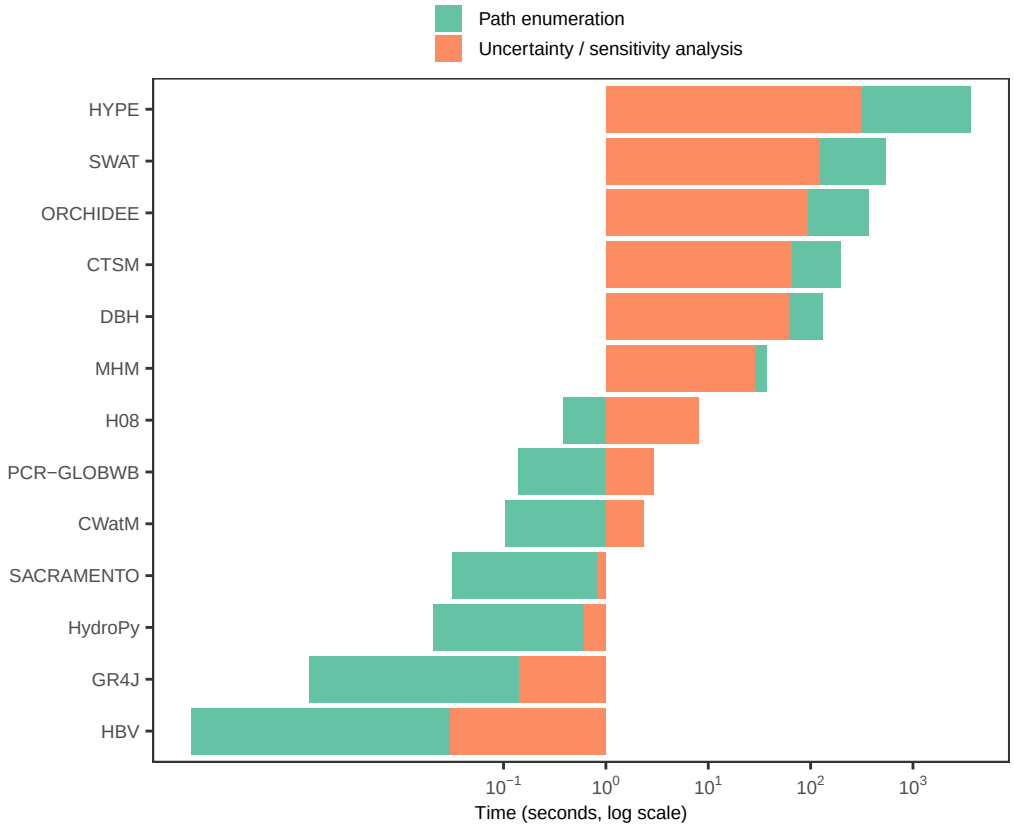
```
cat("\n--- Real model stability summary ---\n")
```

```
##
## --- Real model stability summary ---
```

```
print(real_scalability[!is.na(top1_stability), .(
  median_top1    = round(median(top1_stability), 3),
  min_top1       = round(min(top1_stability), 3),
  max_top1       = round(max(top1_stability), 3),
  median_jaccard = round(median(top10_jaccard), 3),
  min_jaccard    = round(min(top10_jaccard), 3),
  max_jaccard    = round(max(top10_jaccard), 3)
)])
```

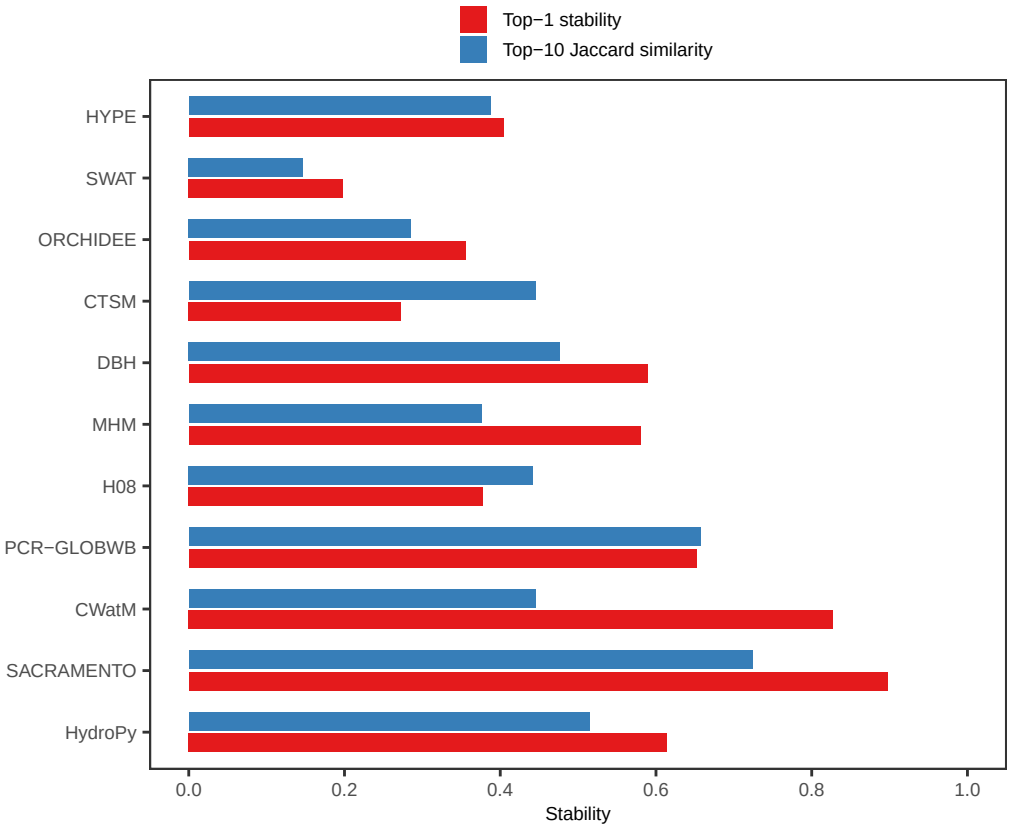
```
##    median_top1 min_top1 max_top1 median_jaccard min_jaccard max_jaccard
##          <num>    <num>    <num>          <num>       <num>       <num>
## 1:        0.58    0.198    0.897          0.445       0.147       0.724
```

```
plot_time_real <- ggplot(time_real_long,
                         aes(model, seconds, fill = stage)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  scale_y_log10(breaks = c(0.1, 1, 10, 100, 1000),
                labels = scales::label_math(10^.x, format = log10)) +
  labs(x = "", y = "Time (seconds, log scale)") +
  scale_fill_brewer(palette = "Set2", name = "",
                    guide = guide_legend(nrow = 2)) +
  theme_AP() +
  theme(legend.position = "top")

plot_time_real
```

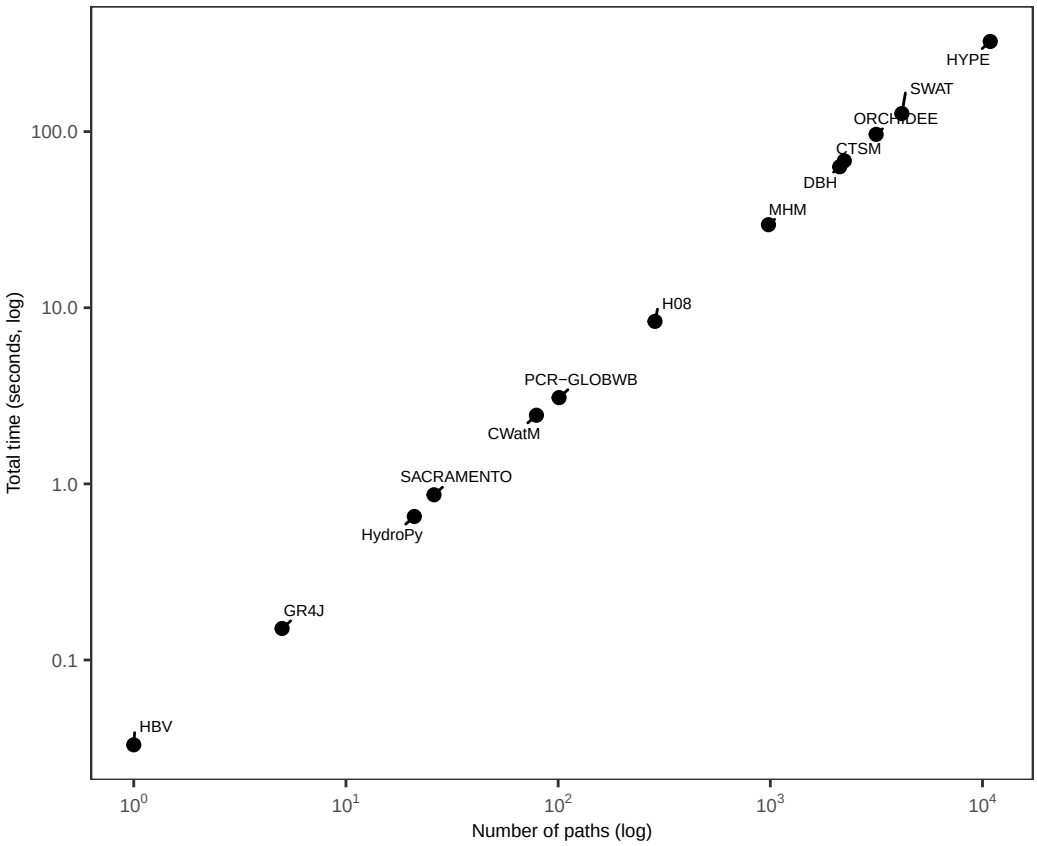



```
plot_stability_real <- ggplot(stability_real_long,
                              aes(model, value, fill = metric)) +
  geom_bar(stat = "identity", position = position_dodge(0.7), width = 0.6) +
  coord_flip() +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(x = "", y = "Stability") +
  scale_fill_brewer(palette = "Set1", name = "",
                    guide = guide_legend(nrow = 2)) +
  theme_AP() +
  theme(legend.position = "top")
```

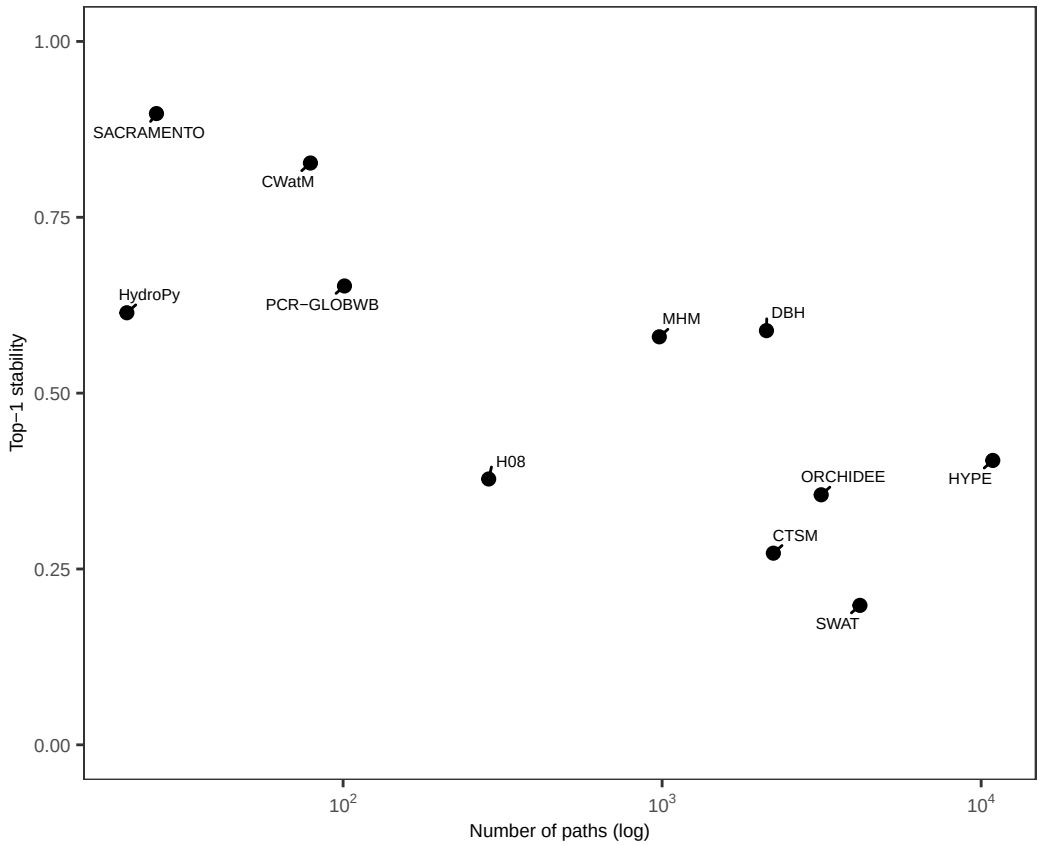
```
plot_stability_real
```



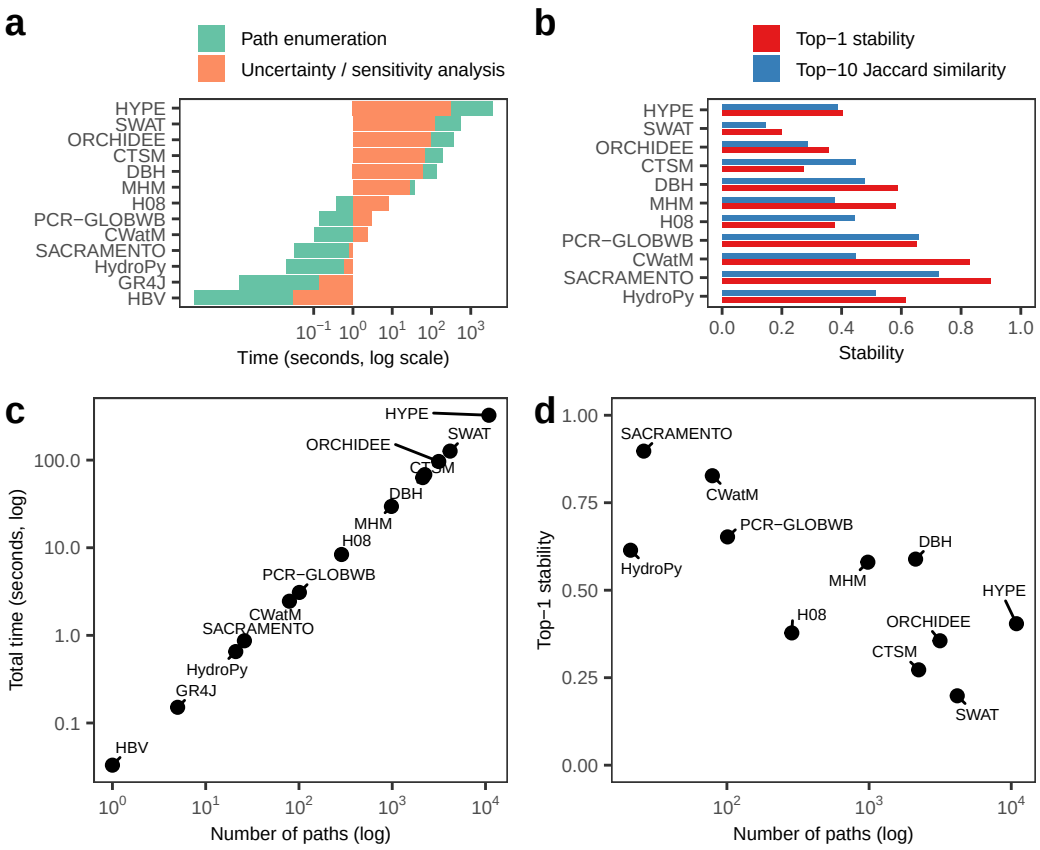

```
plot_time_vs_paths <- real_scalability[!is.na(t_total)] %>%
  ggplot(., aes(n_paths, t_total)) +
  geom_point(size = 2) +
  geom_text_repel(aes(label = model), size = 2, max.overlaps = 20,
                  min.segment.length = 0, seed = 123) +
  scale_x_log10(labels = scales::label_math(10^.x, format = log10)) +
  scale_y_log10() +
  labs(x = "Number of paths (log)", y = "Total time (seconds, log)") +
  theme_AP()

plot_time_vs_paths
```




```
plot_stab_vs_paths <- real_scalability[!is.na(top1_stability)] %>%
  ggplot(., aes(n_paths, top1_stability)) +
  geom_point(size = 2) +
  geom_text_repel(aes(label = model), size = 2, max.overlaps = 20,
                  min.segment.length = 0, seed = 123) +
  scale_x_log10(labels = scales::label_math(10^.x, format = log10)) +
  scale_y_continuous(limits = c(0, 1)) +
  labs(x = "Number of paths (log)", y = "Top-1 stability") +
  theme_AP()

plot_stab_vs_paths
```




```
# E) Merge panels ------------------------------------------------------------

top_row <- plot_grid(plot_time_real, plot_stability_real, ncol = 2,
                     labels = c("a", "b"), align = "h", axis = "tb")
bottom_row <- plot_grid(plot_time_vs_paths, plot_stab_vs_paths, ncol = 2,
                        labels = c("c", "d"), align = "h", axis = "tb")
plot_grid(top_row, bottom_row, ncol = 1, rel_heights = c(0.45, 0.55))
```



4 Analysis of git logs

```
# METRICS AT THE FILE AND FUNCTION LEVEL ######################################

folder <- "./datasets/git_logs"

# Get names of files ----------------------------------------------------------

csv_files <- list.files(path = folder, pattern = "_2\\.csv$", full.names = TRUE)
csv_files
```

```
## [1] "./datasets/git_logs/CTSM_fortran_results-arnald_2.csv"
## [2] "./datasets/git_logs/CTSM_python_results-arnald_2.csv"
## [3] "./datasets/git_logs/CWatM_python_results-arnald_2.csv"
## [4] "./datasets/git_logs/PCR-GLOBWB_python_results-arnald_2.csv"
## [5] "./datasets/git_logs/VIC_fortran_results-arnald_2.csv"
## [6] "./datasets/git_logs/VIC_python_results-arnald_2.csv"
```

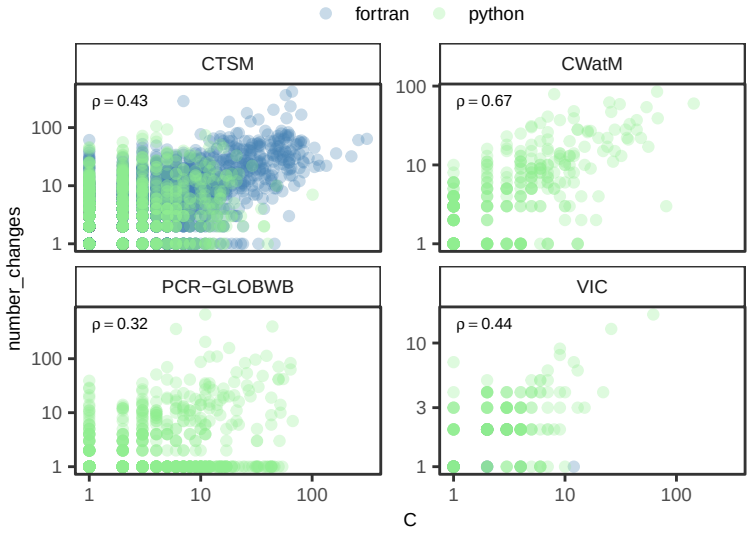
```
# Merge and flatten -----------------------------------------------------------

logs_dt <- lapply(csv_files, fread) %>%
  lapply(., function(x) x[, .(model, language, `function`, cyclomatic_complexity, number_change
                              change_per_days, age_days)]) %>%
```

```
  rbindlist() %>%
  na.omit() %>%
  setnames(., "function", "node")

# Create and run function -----------------------------------------------------

spearman_fun <- function(dt) {

  cor.test(dt[, cyclomatic_complexity],
           dt[, number_changes], method = "spearman")$estimate[[1]]
}

rho_values <- logs_dt[, .(rho = spearman_fun(.SD)), model]
```

```
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
## Warning in cor.test.default(dt[, cyclomatic_complexity], dt[, number_changes],
## : Cannot compute exact p-value with ties
```

```
# Create dt to position labels ------------------------------------------------

pos_df <- logs_dt %>%
  filter(is.finite(cyclomatic_complexity), is.finite(number_changes)) %>%
  group_by(model) %>%
  summarise(x = min(cyclomatic_complexity, na.rm = TRUE),
            y = max(number_changes, na.rm = TRUE), .groups = "drop")

# join rho and make label text ------------------------------------------------

lab_df <- rho_values %>%
  left_join(pos_df, by = "model") %>%
  mutate(label = ifelse(is.na(rho), "rho==NA", paste0("rho==", round(rho, 2))))

# Plot ------------------------------------------------------------------------

plot_logs <- logs_dt %>%
  ggplot(aes(cyclomatic_complexity, number_changes, color = language)) +
  geom_point(alpha = 0.3) +
  scale_color_manual(values = color_languages, name = "") +
  scale_x_log10() +
  scale_y_log10() +
  labs(x = expression(C), y = "number_changes") +
  facet_wrap(~model, scales = "free_y") +
  geom_text(data = lab_df %>% na.omit(),
```

```
            aes(x = x, y = y, label = label), inherit.aes = FALSE,
            hjust = -0.05, vjust = 1.2, size = 2, parse = TRUE) +
  coord_cartesian(clip = "off") +
  theme_AP() +
  theme(legend.position = "top")

plot_logs
```




```
# ARE REFACTORING ACTIONS MORE COMMON IN RISKY PATHS? ############################

models_to_check <- unique(logs_dt$model)

# Extract name of functions in top ten risky paths per model --------------------

list_nodes <- paths_all %>%
  data.table() %>%
  .[order(-path_risk_score), .SD[1:10], model] %>%
  .[model %in% models_to_check] %>%
  na.omit() %>%
  .[, .(node = unlist(path_nodes)), model]

# keep only unique risk nodes
risk_nodes <- unique(list_nodes[, .(model, node)])
risk_nodes[, high_risk_path := TRUE]

# Filter out and keep only models that have log data ----------------------------

logs_dt <- logs_dt[model %in% models_to_check]

# left join into logs_dt --------------------------------------------------------

logs_dt[risk_nodes, high_risk_path:= i.high_risk_path,
```

```
        on = .(model, node)]

# Turn NAs into FALSE ------------------------------------------------------

logs_dt[is.na(high_risk_path), high_risk_path := FALSE]

# We now quantify effect size using the common-language statistic A,
# defined as the probability that a randomly selected high-risk function
# has higher churn than a randomly selected non-risk function. -----------------

wilcox_dt <- logs_dt[is.finite(number_changes),
                     {
                       x <- number_changes[high_risk_path]
                       y <- number_changes[!high_risk_path]

                       n_x <- length(x)
                       n_y <- length(y)

                       if (n_x >= 3L && n_y >= 3L) {

                         wt <- wilcox.test(x, y, alternative = "greater", exact = FALSE)

                         # Wilcoxon rank-sum statistic (sum of ranks for x)--------------------
                         W <- as.numeric(wt$statistic)

                         # Convert to Mann-Whitney U ----------------------------------------
                         U <- W - n_x * (n_x + 1) / 2

                         # Common-language effect size (A statistic) ------------------------
                         A <- U / (n_x * n_y)

                         .(
                           p_value = wt$p.value,
                           A_common_language = A,
                           n_risk = n_x,
                           n_nonrisk = n_y,
                           median_risk = median(x),
                           median_nonrisk = median(y)
                         )

                       } else {

                         .(
                           p_value = NA_real_,
                           A_common_language = NA_real_,
                           n_risk = n_x,
                           n_nonrisk = n_y,
```

```
                        median_risk = if (n_x) median(x) else NA_real_,
                        median_nonrisk = if (n_y) median(y) else NA_real_
                      )
                    }
                  },
                  model
]


wilcox_dt
```

```
##         model      p_value A_common_language n_risk n_nonrisk median_risk
##        <char>        <num>             <num>  <int>     <int>       <num>
## 1:       CTSM 2.649807e-06         0.6783021     50      3363        10.0
## 2:      CWatM 1.097483e-03         0.6967666     16       317        10.0
## 3: PCR-GLOBWB 1.332338e-02         0.5935438     24       859         1.5
## 4:        VIC           NA                NA      0       203          NA
##    median_nonrisk
##             <num>
## 1:              4
## 2:              3
## 3:              1
## 4:              2
```

```
# FUNCTIONS TO SELECT THE TOP TEN RISKY PATHS PER MODEL AND
# PRINT THEM OUT FOR LATEX ###########################################################

# Top ten per model ------------------------------------------------------------

tmp <- paths_all[risk_function == "additive"] %>%
  .[order(-path_risk_score), .SD[1:10], model] %>%
  .[, .(model, path_str)] %>%
  na.omit() %>%
  split(., .$model)


to_tex_list_fun(tmp)

# Random sample from bottom 20% per model --------------------------------------

set.seed(seed)

tmp2 <- paths_additive[, {n_bot <- ceiling(.N * 0.2)
.SD[order(path_risk_score)] %>%
    .[sample(seq_len(n_bot), size = min(5, n_bot))]
  }, model] %>%
  .[, .(model, path_str)] %>%
  na.omit() %>%
```

```
  split(., .$model)

to_tex_list_fun(tmp2)
```

5 Session information

```
# SESSION INFORMATION ############################################################

sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framew
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] tools     parallel  stats     graphics  grDevices utils     datasets
## [8] methods   base
##
## other attached packages:
##  [1] softwareRisk_0.2.0  benchmarkme_1.0.8   sensobol_1.1.6
##  [4] ggraph_2.2.2        foreach_1.5.2       igraph_2.2.1
##  [7] tidygraph_1.3.1     here_1.0.2          tidytext_0.4.3
## [10] ggrepel_0.9.6       readxl_1.4.5        cowplot_1.2.0.9000
## [13] scales_1.4.0        openxlsx_4.2.8      lubridate_1.9.4
## [16] forcats_1.0.1       stringr_1.6.0       dplyr_1.1.4
## [19] purrr_1.2.1         readr_2.2.0         tidyr_1.3.2
## [22] tibble_3.3.1        ggplot2_4.0.1.9000  tidyverse_2.0.0
## [25] data.table_1.18.2.1
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.1      viridisLite_0.4.2     farver_2.1.2
##  [4] viridis_0.6.5         S7_0.2.1              fastmap_1.2.0
##  [7] tweenr_2.0.3          janeaustenr_1.0.0     digest_0.6.39
## [10] timechange_0.3.0      lifecycle_1.0.5       tokenizers_0.3.0
## [13] magrittr_2.0.4        compiler_4.5.2        rlang_1.1.7
## [16] utf8_1.2.6            yaml_2.3.12           knitr_1.51
## [19] labeling_0.4.3        graphlayouts_1.2.2    RColorBrewer_1.1-3
## [22] withr_3.0.2           grid_4.5.2            polyclip_1.10-7
## [25] iterators_1.0.14      MASS_7.3-65           tinytex_0.58
## [28] cli_3.6.5             rmarkdown_2.30        generics_0.1.4
## [31] RcppParallel_5.1.11-1 otel_0.2.0            rstudioapi_0.17.1
## [34] httr_1.4.8            tzdb_0.5.0            cachem_1.1.0
```

```
## [37] ggforce_0.5.0         cellranger_1.1.0      vctrs_0.7.1
## [40] Matrix_1.7-4          hms_1.1.4             ineq_0.2-13
## [43] glue_1.8.0            benchmarkmeData_1.0.4 codetools_0.2-20
## [46] rngWELL_0.10-10       stringi_1.8.7         gtable_0.3.6
## [49] randtoolbox_2.0.5     pillar_1.11.1         htmltools_0.5.9
## [52] R6_2.6.1              zigg_0.0.2            Rdpack_2.6.4
## [55] doParallel_1.0.17     rprojroot_2.1.1       evaluate_1.0.5
## [58] lattice_0.22-7        rbibutils_2.4         SnowballC_0.7.1
## [61] Rfast_2.1.5.2         memoise_2.0.1         Rcpp_1.1.1
## [64] zip_2.3.3             gridExtra_2.3         xfun_0.55
## [67] pkgconfig_2.0.3
```

```
## Return the machine CPU ------------------------------------------------------

cat("Machine:     "); print(get_cpu()$model_name)
```

```
## Machine:
## [1] "Apple M1 Max"
```

```
## Return number of true cores -------------------------------------------------

cat("Num cores:   "); print(parallel::detectCores(logical = FALSE))
```

```
## Num cores:
## [1] 10
```

```
## Return number of threads ----------------------------------------------------

cat("Num threads: "); print(parallel::detectCores(logical = FALSE))
```

```
## Num threads:
## [1] 10
```